
Sprint review #11

HyDE feed recommendation

API Spec Link :

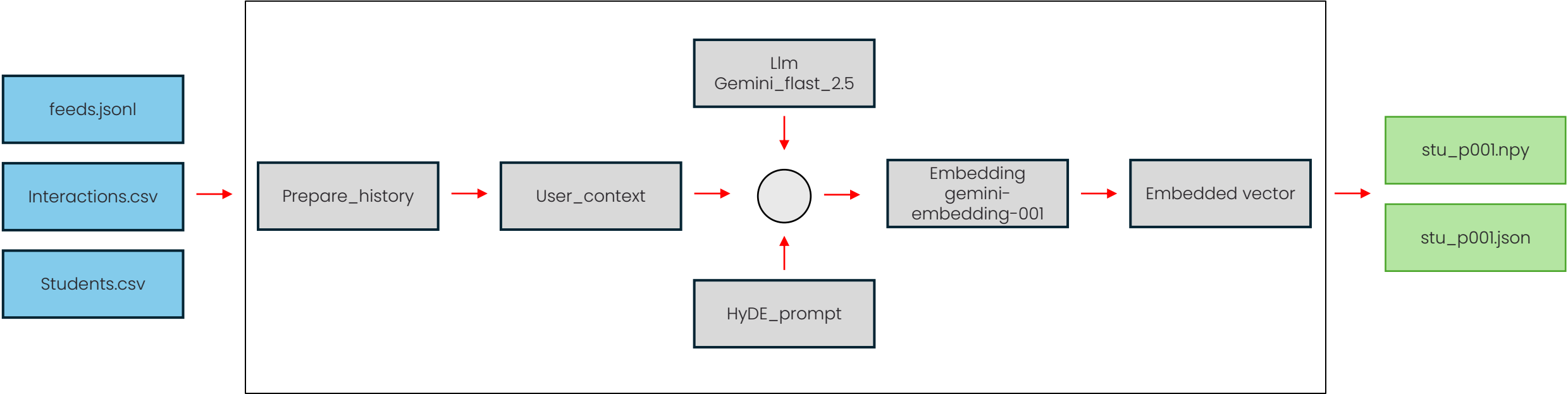
Orchestrator :

API core evaluation link : <https://hyderecomment-service-v1-1-0-du7yhkyaqg-as.a.run.app/docs#/>

Github : <https://github.com/TunKedsaro/Hyde>

QA : <https://humanintelligence.atlassian.net/wiki/spaces/CPM/pages/134611027/QA+Test+Plan+CV+Evaluation+Pipeline>

HyDE Feed Recommendation - System Overview

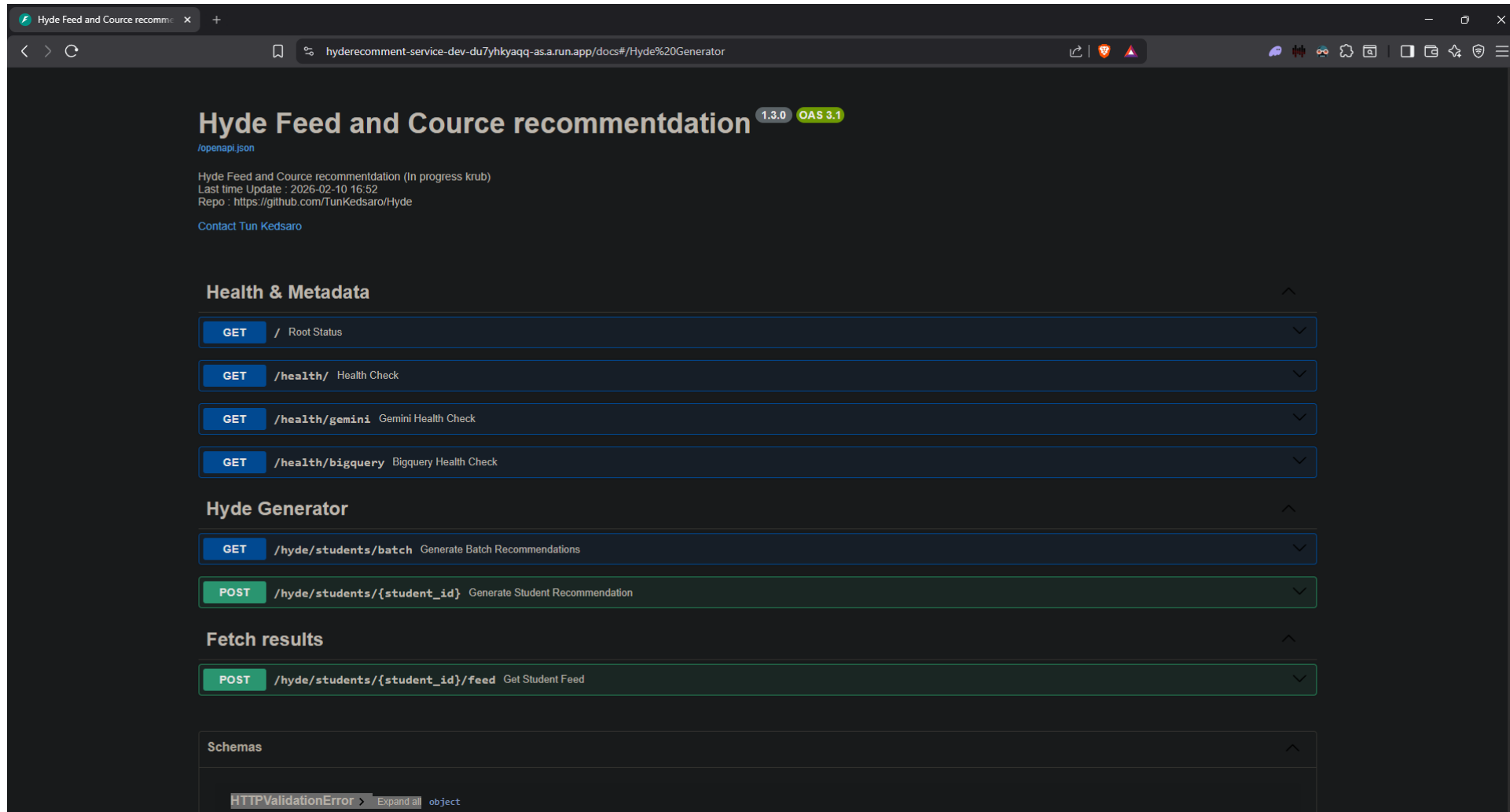


Legend



Process

HyDE Feed Recommendation - System Overview



HyDE recommendation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation - Generate HyDE bundle for a single student

The image shows two side-by-side browser windows. The left window displays the Google Cloud Storage console for a bucket named 'hyde-datalake'. The right window shows the HyDE Generator API interface.

Google Cloud Storage Console (Left Window):

- Page Title: Cloud Storage
- Bucket Name: hyde-datalake
- Location: asia-southeast1 (Singapore)
- Storage class: Standard
- Public access: Not public
- Protection: Soft delete
- Tab: Objects
- Folder browser: hyde-datalake
- Subfolders: stu_p001/, stu_p100/
- Filter by name prefix only
- Table of objects:

Name
stu_p001/
stu_p100/

HyDE Generator API Interface (Right Window):

Health & Metadata

- GET / Root Status
- GET /health/ Health Check
- GET /health/gemini Gemini Health Check
- GET /health/bigquery Bigquery Health Check

Hyde Generator

- GET /hyde/students/batch Generate Batch Recommendations
- POST /hyde/students/{student_id} Generate Student Recommendation

API:02 Connectivity health check for Gemini LLM service. Verifies API availability and measures round-trip response latency.

Parameters

Name	Description
student_id * required	
any (path)	stu_p002

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'https://hyderecommend-service-dev-du7yhkyaqg-as-a.run.app/hyde/students/stu_p001' \
  -H 'accept: application/json' \
  -d ''
```

3:00 PM
2/10/2026

HyDE recommendation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation - Generate HyDE bundle for a single student

The image shows two side-by-side browser windows. The left window displays the Google Cloud Storage console for a bucket named 'hyde-datalake'. The 'Objects' tab is active, showing a folder browser with subfolders 'stu_p001/', 'stu_p002/', and 'stu_p100/'. The right window shows a REST client interface for a POST request to the endpoint '/hyde/students/{student_id}'. The request body is a JSON object with 'student_id' set to 'stu_p002'. The response is a 200 status code with a JSON body containing 'student_id': 'stu_p002' and 'response': 'Complete'.

Google Cloud Storage Console (Left Window):

- Bucket: hyde-datalake
- Location: asia-southeast1 (Singapore)
- Storage class: Standard
- Public access: Not public
- Protection: Soft delete
- Objects tab:stu_p001/
- stu_p002/
- stu_p100/

REST Client (Right Window):

POST /hyde/students/{student_id} Generate Student Recommendation

API:02 Connectivity health check for Gemini LLM service. Verifies API availability and measures round-trip response latency.

Parameters

Name	Description
student_id * required	any (path)

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://hyderecommend-service-dev-du7yhkyaqg-as.a.run.app/hyde/students/stu_p002' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

https://hyderecommend-service-dev-du7yhkyaqg-as.a.run.app/hyde/students/stu_p002

Server response

Code	Details
200	Response body <pre>{ "student_id": "stu_p002", "response": "Complete" }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: * alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000 content-length: 47</pre>

HyDE recommenddation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation - Generate HyDE bundle for a single student

The image shows two side-by-side browser windows. The left window displays the Google Cloud Storage console for a bucket named 'hyde-datalake'. The bucket is located in 'asia-southeast1 (Singapore)' with a 'Standard' storage class, 'Not public' access, and 'Soft delete' protection. The 'Objects' tab is selected, showing a 'Folder browser' with a table that currently has 'No rows to display'. A large circular icon with a plus sign and a document symbol is centered in the main area, with the text: 'Your bucket is ready. Just add data. Drop files and folders here or use the upload button. To move a lot of data from another bucket or cloud storage provider, create a transfer job.' A notification at the bottom states '12 folders deleted'. The right window shows the 'Hyde Feed and Course recommendation' API interface. It includes a version indicator '1.3.0 OAS 3.1', a link to '/openapi.json', and a description: 'Hyde Feed and Course recommendation (In progress krub)'. It also lists the 'Last time Update : 2026-02-10 16:52' and the 'Repo : https://github.com/TunKedsaro/Hyde'. The 'Health & Metadata' section contains four GET endpoints: '/ Root Status', '/health/ Health Check', '/health/gemini Gemini Health Check', and '/health/bigquery Bigquery Health Check'. The 'Hyde Generator' section contains two endpoints: a GET endpoint '/hyde/students/batch Generate Batch Recommendations' and a POST endpoint '/hyde/students/{student_id} Generate Student Recommendation'. The 'Fetch results' section contains a POST endpoint '/hyde/students/{student_id}/feed Get Student Feed'. A 'Schemas' section is partially visible at the bottom.

HyDE recommenddation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation - Generate HyDE bundle for a single student

The image shows two side-by-side browser windows. The left window displays the Google Cloud Storage console for a bucket named 'hyde-datalake'. The right window shows the 'Hyde Feed and Course recommendation' API interface.

Google Cloud Storage Console (Left Window):

- Bucket: hyde-datalake
- Location: asia-southeast1 (Singapore)
- Storage class: Standard
- Public access: Not public
- Protection: Soft delete
- Tab: Objects
- Folder browser: hyde-datalake
- Subfolders: stu_p000/, stu_p001/, embedding/, hyde/, metadata/, stu_p002/, stu_p003/, stu_p004/, stu_p005/, stu_p006/, stu_p007/, stu_p008/, stu_p009/, stu_p010/, stu_p100/
- Table of objects in embedding/ folder:

Name	Size	Type
embedding01.npy	3.2 KB	ap
embedding02.npy	3.2 KB	ap
embedding03.npy	3.2 KB	ap
embedding04.npy	3.2 KB	ap
embedding05.npy	3.2 KB	ap

HyDE Feed and Course recommendation API (Right Window):

- Version: 1.3.0 OAS 3.1
- Endpoint: /openapi.json
- Description: Hyde Feed and Course recommendation (In progress krub)
Last time Update : 2026-02-10 16:52
Repo : https://github.com/TunKedsaro/Hyde
Contact Tun Kedsaro
- Health & Metadata:
 - GET / Root Status
 - GET /health/ Health Check
 - GET /health/gemini Gemini Health Check
 - GET /health/bigquery Bigquery Health Check
- Hyde Generator:
 - GET /hyde/students/batch Generate Batch Recommendations
 - API 2.1 : Generate recommendations for all students (batch)
 - Parameters: No parameters
 - Buttons: Execute, Clear, Cancel

HyDE recommendation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation – Fetch result

Fetch results

POST /hyde/students/{student_id}/feed Get Student Feed

API 3.1 : Fetch generated HyDE feed results and embeddings

Parameters Cancel

Name	Description
student_id * required	
any	stu_p001
(path)	

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://hyderecommend-service-dev-du7yhkyaqg-as.a.run.app/hyde/students/stu_p001/feed' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
https://hyderecommend-service-dev-du7yhkyaqg-as.a.run.app/hyde/students/stu_p001/feed
```

Server response

Code	Details
200	Response body <pre>{ "student_id": "stu_p001", "status": "hyde-datalake /\n - stu_p001 /\n - metadata folder /\n - metadata.json /\n - hyde folder /\n - stu_p001/hyde/hyde_text01.txt /\n - stu_p001/hyde/hyde_text02.txt /\n - stu_p001/hyde/hyde_text03.txt /\n - stu_p001/hyde/hyde_text04.txt /\n - stu_p001/hyde/hyde_text05.txt /\n - embedding folder /\n - stu_p001/embedding/embedding01.npy /\n - stu_p001/embedding/embedding02.npy /\n - stu_p001/embedding/embedding03.npy /\n - stu_p001/embedding/embedding04.npy /\n - stu_p001/embedding/embedding05.npy /\n", "metadata": { "student_id": "stu_p001", "current_status": "student3yr", </pre>

HyDE recommendation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation – Fetch result

The screenshot shows a web browser window with the URL `hyderecommend-service`. The page title is "Fetch results". The API endpoint is `POST /hyde/students/{student_id}`. The parameters section shows `student_id` with the value `stu_p001`. The response body is a JSON object:

```
{
  "student_id": "stu_p001",
  "status": "\"hyde-datalake /\n|- stu_p001 /\n|- metadata folder /\n|- metadata.json /\n|- hyde folder /\n|- stu_p001/hyde/hyde_text01.txt /\n|- stu_p001/hyde/hyde_text02.txt /\n|- stu_p001/hyde/hyde_text03.txt /\n|- stu_p001/hyde/hyde_text04.txt /\n|- stu_p001/hyde/hyde_text05.txt /\n|- embedding folder /\n|- stu_p001/embedding/embedding01.npy /\n|- stu_p001/embedding/embedding02.npy /\n|- stu_p001/embedding/embedding03.npy /\n|- stu_p001/embedding/embedding04.npy /\n|- stu_p001/embedding/embedding05.npy /\n\"",
  "metadata": {
    "student_id": "stu_p001",
    "current_status": "student3yr",
    "education_level": "bachelor",
    "education_major": "วิทยาการคอมพิวเตอร์",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "hyde": {
    "hyde_context1": "ไอเดียโปรเจกต์ทำพอร์ต Data Analyst ด้วย Python SQL",
    "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python",
    "hyde_context3": "",
    "hyde_context4": "เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลยอดนิยม",
    "hyde_context5": "ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst"
  },
  "embedded_vector": {
    "embedding_vector1": [
      0.023251675069332123,
      -0.014571324922144413,
      0.01352651696652174
    ],
    "embedding_vector2": [
      0.007405612617731094,
      -0.03062768094241619,
      0.0034155375324189663
    ],
    "embedding_vector3": [],
    "embedding_vector4": [
      -0.011106484569609165,
      0.014421403408050537,
      -0.0009799925610423088
    ],
    "embedding_vector5": [
      0.0059158275835216045,
      -0.007175480481237173,
      0.007349800784140825
    ]
  }
}
```

HyDE recommendation core engine API

GCS / hyde-datalake (Test)

HyDE Feed Recommendation - System Overview

Overview

```
{
  "student_id": "stu_p001",
  "status": "\"\"hyde-datalake /\n|- stu_p001 /\n|- metadata folder /\n|- metadata.json /\n|- hyde folder /\n|- stu_p001/hyde/hyde_text01.txt /\n|- stu_p001/hyde/hyde_text02.txt /\n|- stu_p001/hyde/hyde_text03.txt x\n|- stu_p001/hyde/hyde_text04.txt /\n|- stu_p001/hyde/hyde_text05.txt /\n|- embedding folder /\n|- stu_p001/embedding/embedding01.npy /\n|- stu_p001/embedding/embedding02.npy /\n|- stu_p001/embedding/embedding03.npy x\n|- stu_p001/embedding/embedding04.npy /\n|- stu_p001/embedding/embedding05.npy /\n\"",
  "metadata": {
    "student_id": "stu_p001",
    "current_status": "student3yr",
    "education_level": "bachelor",
    "education_major": "วิทยาการคอมพิวเตอร์",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "hyde": {
    "hyde_context1": "ไอเดียโปรเจกต์ทำพอร์ต Data Analyst ด้วย Python SQL",
    "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python",
    "hyde_context3": "",
    "hyde_context4": "เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลออนไลน์",
    "hyde_context5": "ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst"
  },
  "embedded_vector": {
    "embedding_vector1": [
      0.023251675069332123,
      -0.014571324922144413,
      0.01352651696652174
    ],
    "embedding_vector2": [
      0.007405612617731094,
      -0.03062768094241619,
      0.0034155375324189663
    ],
    "embedding_vector3": [],
    "embedding_vector4": [
      -0.011106484569609165,
      0.014421403408050537,
      -0.0009799925610423088
    ],
    "embedding_vector5": [
      0.0059158275835216045,
      -0.007175480481237173,
      0.007349800784140825
    ]
  }
}
```

HyDE Feed Recommendation - System Overview

Overview

```
{
  "student_id": "stu_p001",
  "status": "\"hyde-datalake /\n|- stu_p001 /\n|- metadata folder /\n|- metadata.json /\n|- hyde folder /\n|- stu_p001/hyde/hyde_text01.txt /\n|- stu_p001/hyde/hyde_text02.txt /\n|- stu_p001/hyde/hyde_text03.txt x\n|- stu_p001/hyde/hyde_text04.txt /\n|- stu_p001/hyde/hyde_text05.txt /\n|- embedding folder /\n|- stu_p001/embedding/embedding01.npy /\n|- stu_p001/embedding/embedding02.npy /\n|- stu_p001/embedding/embedding03.npy x\n|- stu_p001/embedding/embedding04.npy /\n|- stu_p001/embedding/embedding05.npy /\n\"",
  "metadata": {
    "student_id": "stu_p001",
    "current_status": "student3yr",
    "education_level": "bachelor",
    "education_major": "วิทยาการคอมพิวเตอร์",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "hyde": {
    "hyde_context1": "โลเคชันโปรเจกต์ทำพอร์ต Data Analyst ด้วย Python SQL",
    "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python",
    "hyde_context3": "",
    "hyde_context4": "เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลออนไลน์",
    "hyde_context5": "ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst"
  },
  "embedded_vector": {
    "embedding_vector1": [
      0.023251675069332123,
      -0.014571324922144413,
      0.01352651696652174
    ],
    "embedding_vector2": [
      0.007405612617731094,
      -0.03062768094241619,
      0.0034155375324189663
    ],
    "embedding_vector3": [],
    "embedding_vector4": [
      -0.011106484569609165,
      0.014421403408050537,
      -0.0009799925610423088
    ],
    "embedding_vector5": [
      0.0059158275835216045,
      -0.007175480481237173,
      0.007349800784140825
    ]
  }
}
```

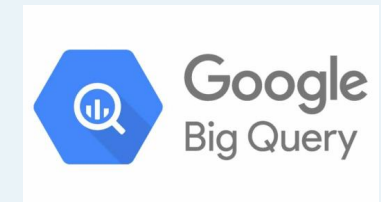
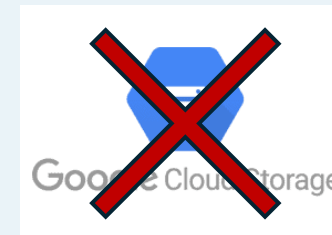
```
hyde-datalake /
|- stu_p001 /
  |- metadata folder /
    |- metadata.json /
  |- hyde folder /
    |- stu_p001/hyde/hyde_text01.txt /
    |- stu_p001/hyde/hyde_text02.txt /
    |- stu_p001/hyde/hyde_text03.txt x
    |- stu_p001/hyde/hyde_text04.txt /
    |- stu_p001/hyde/hyde_text05.txt /
  |- embedding folder /
    |- stu_p001/embedding/embedding01.npy /
    |- stu_p001/embedding/embedding02.npy /
    |- stu_p001/embedding/embedding03.npy x
    |- stu_p001/embedding/embedding04.npy /
    |- stu_p001/embedding/embedding05.npy /
```

HyDE Feed Recommendation - System Overview

Overview

```
{
  "student_id": "stu_p001",
  "status": "\"hyde-datalake /\n|- stu_p001 /\n|- metadata folder /\n|- metadata.json /\n|- hyde folder /\n|- stu_p001/hyde/hyde_text01.txt /\n|- stu_p001/hyde/hyde_text02.txt /\n|- stu_p001/hyde/hyde_text03.txt x\n|- stu_p001/hyde/hyde_text04.txt /\n|- stu_p001/hyde/hyde_text05.txt /\n|- embedding folder /\n|- stu_p001/embedding/embedding01.npy /\n|- stu_p001/embedding/embedding02.npy /\n|- stu_p001/embedding/embedding03.npy x\n|- stu_p001/embedding/embedding04.npy /\n|- stu_p001/embedding/embedding05.npy /\n\"",
  "metadata": {
    "student_id": "stu_p001",
    "current_status": "student3yr",
    "education_level": "bachelor",
    "education_major": "วิทยาการคอมพิวเตอร์",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "hyde": {
    "hyde_context1": "โลเคชันโปรแกรมทำพอร์ต Data Analyst ด้วย Python SQL",
    "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python",
    "hyde_context3": "",
    "hyde_context4": "เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลออนไลน์",
    "hyde_context5": "ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst"
  },
  "embedded_vector": {
    "embedding_vector1": [
      0.023251675069332123,
      -0.014571324922144413,
      0.01352651696652174
    ],
    "embedding_vector2": [
      0.007405612617731094,
      -0.03062768094241619,
      0.0034155375324189663
    ],
    "embedding_vector3": [],
    "embedding_vector4": [
      -0.011106484569609165,
      0.014421403408050537,
      -0.0009799925610423088
    ],
    "embedding_vector5": [
      0.0059158275835216045,
      -0.007175480481237173,
      0.007349800784140825
    ]
  }
}
```

```
{'student_id': 'stu_p001',
 'current_status': 'student3yr',
 'education_level': 'bachelor',
 'education_major': 'วิทยาการคอมพิวเตอร์',
 'target_roles': 'Data Analyst',
 'timezone': 'UTC',
 'model_name': 'gemini-2.5-flash',
 'max_output_tokens': 2048,
 'feed_text_max_chars': 240,
 'temperature': 0.2}
```



HyDE Feed Recommendation - System Overview

Overview

```
{
  "student_id": "stu_p001",
  "status": "\"hyde-datalake /\n|- stu_p001 /\n|- metadata folder /\n|- metadata.json /\n|- hyde folder /\n|- stu_p001/hyde/hyde_text01.txt /\n|- stu_p001/hyde/hyde_text02.txt /\n|- stu_p001/hyde/hyde_text03.txt x\n|- stu_p001/hyde/hyde_text04.txt /\n|- stu_p001/hyde/hyde_text05.txt /\n|- embedding folder /\n|- stu_p001/embedding/embedding01.npy /\n|- stu_p001/embedding/embedding02.npy /\n|- stu_p001/embedding/embedding03.npy x\n|- stu_p001/embedding/embedding04.npy /\n|- stu_p001/embedding/embedding05.npy /\n\"",
  "metadata": {
    "student_id": "stu_p001",
    "current_status": "student3yr",
    "education_level": "bachelor",
    "education_major": "วิทยาการคอมพิวเตอร์",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "hyde": {
    "hyde_context1": "ไอเดียโปรเจกต์ทำพอร์ต Data Analyst ด้วย Python SQL",
    "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python",
    "hyde_context3": "",
    "hyde_context4": "เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลยอดนิยม",
    "hyde_context5": "ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst"
  },
  "embedded_vector": {
    "embedding_vector1": [
      0.023251675069332123,
      -0.014571324922144413,
      0.01352651696652174
    ],
    "embedding_vector2": [
      0.007405612617731094,
      -0.03062768094241619,
      0.0034155375324189663
    ],
    "embedding_vector3": [],
    "embedding_vector4": [
      -0.011106484569609165,
      0.014421403408050537,
      -0.0009799925610423088
    ],
    "embedding_vector5": [
      0.0059158275835216045,
      -0.007175480481237173,
      0.007349800784140825
    ]
  }
}
```

```
{
  'hyde_context1': 'ไอเดียโปรเจกต์ทำพอร์ต Data Analyst ด้วย Python SQL',
  'hyde_context2': 'เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python',
  'hyde_context3': '',
  'hyde_context4': 'เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลยอดนิยม',
  'hyde_context5': 'ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst'
}
```

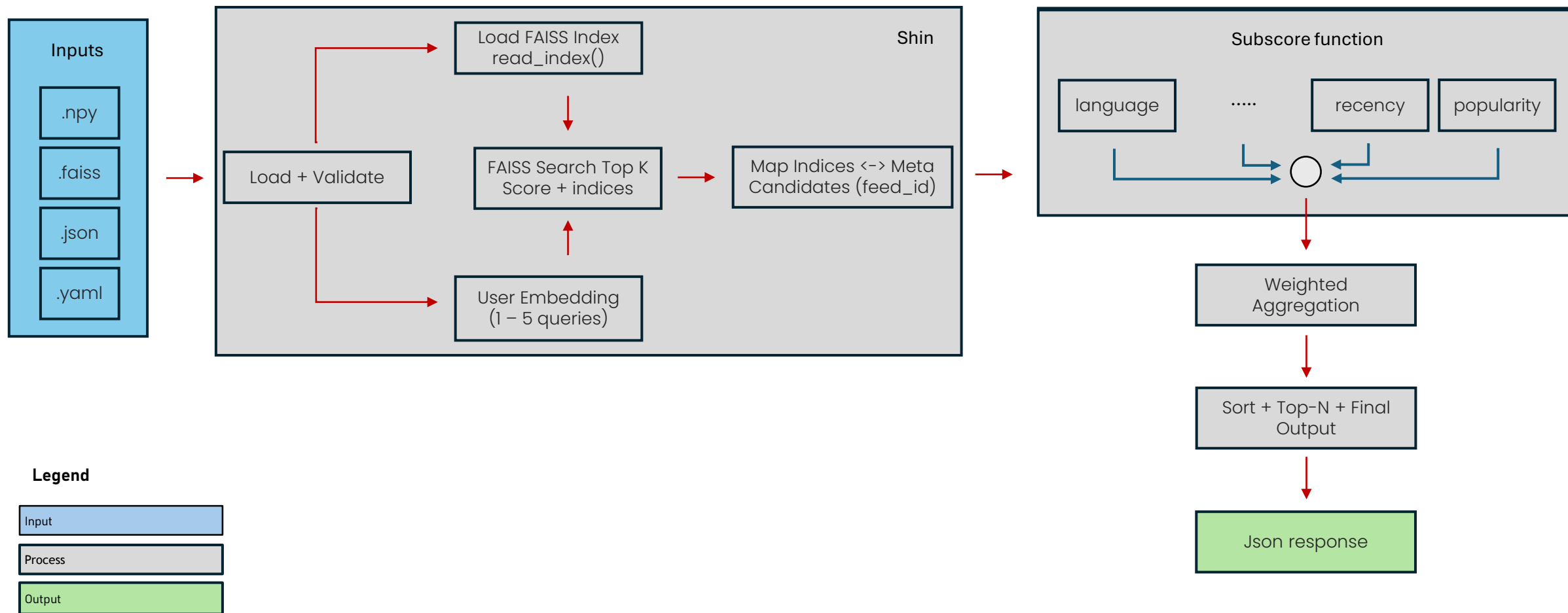

HyDE Feed Recommendation - System Overview

Overview

```
{
  "student_id": "stu_p001",
  "status": "\"hyde-datalake /\n|- stu_p001 /\n|- metadata folder /\n|- metadata.json /\n|- hyde folder /\n|- stu_p001/hyde/hyde_text01.txt /\n|- stu_p001/hyde/hyde_text02.txt /\n|- stu_p001/hyde/hyde_text03.txt x\n|- stu_p001/hyde/hyde_text04.txt /\n|- stu_p001/hyde/hyde_text05.txt /\n|- embedding folder /\n|- stu_p001/embedding/embedding01.npy /\n|- stu_p001/embedding/embedding02.npy /\n|- stu_p001/embedding/embedding03.npy x\n|- stu_p001/embedding/embedding04.npy /\n|- stu_p001/embedding/embedding05.npy /\n\"",
  "metadata": {
    "student_id": "stu_p001",
    "current_status": "student3yr",
    "education_level": "bachelor",
    "education_major": "วิทยาการคอมพิวเตอร์",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "hyde": {
    "hyde_context1": "โหลดโปรเจกต์ทำพอร์ต Data Analyst ด้วย Python SQL",
    "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python",
    "hyde_context3": "",
    "hyde_context4": "เครื่องมือสร้าง Dashboard วิเคราะห์ข้อมูลออนไลน์",
    "hyde_context5": "ทักษะการสื่อสารนำเสนอข้อมูลสำหรับ Data Analyst"
  },
  "embedded_vector": {
    "embedding_vector1": [
      0.023251675069332123,
      -0.014571324922144413,
      0.01352651696652174
    ],
    "embedding_vector2": [
      0.007405612617731094,
      -0.03062768094241619,
      0.0034155375324189663
    ],
    "embedding_vector3": [],
    "embedding_vector4": [
      -0.011106484569609165,
      0.014421403408050537,
      -0.0009799925610423088
    ],
    "embedding_vector5": [
      0.0059158275835216045,
      -0.007175480481237173,
      0.007349800784140825
    ]
  }
}
```

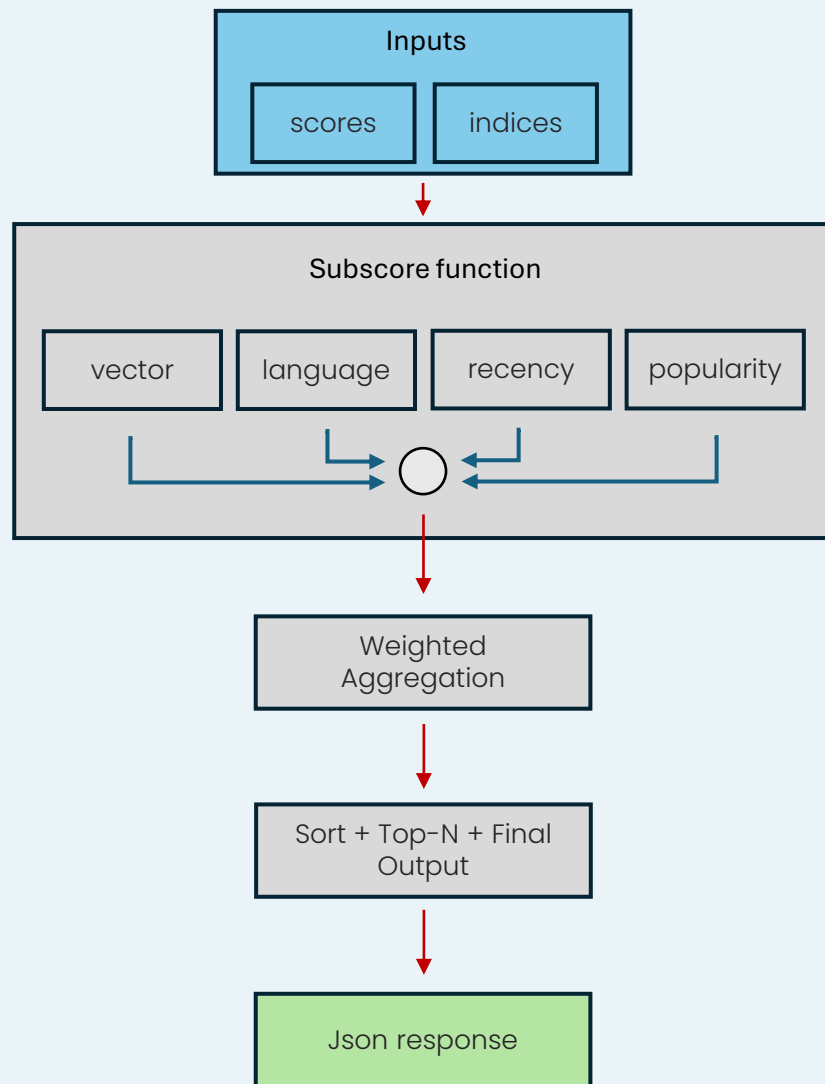
```
"embedded_vector": {
  "embedding_vector1": [
    0.023251675069332123,
    -0.014571324922144413,
    0.01352651696652174
  ],
  "embedding_vector2": [
    0.007405612617731094,
    -0.03062768094241619,
    0.0034155375324189663
  ],
  "embedding_vector3": [],
  "embedding_vector4": [
    -0.011106484569609165,
    0.014421403408050537,
    -0.0009799925610423088
  ],
  "embedding_vector5": [
    0.0059158275835216045,
    -0.007175480481237173,
    0.007349800784140825
  ]
}
```

HyDE Feed Recommendation - System Overview



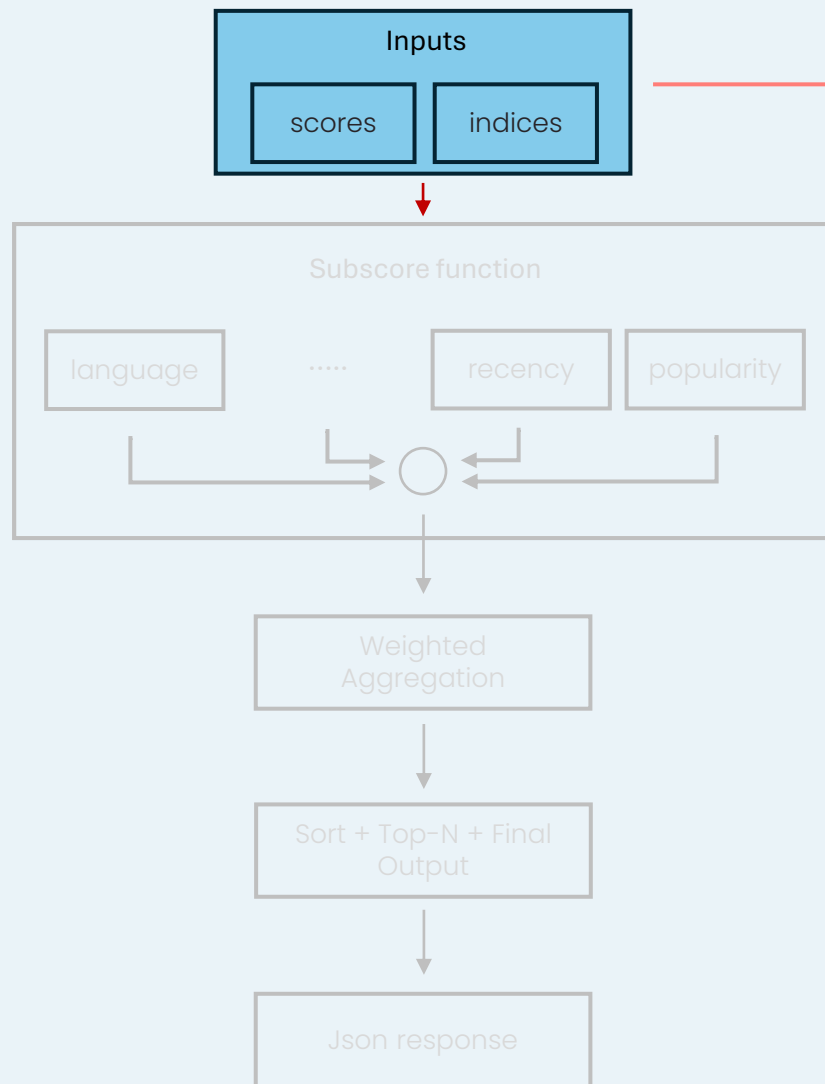
HyDE Feed Recommendation - System Overview

Overview



HyDE Feed Recommendation - Subscore

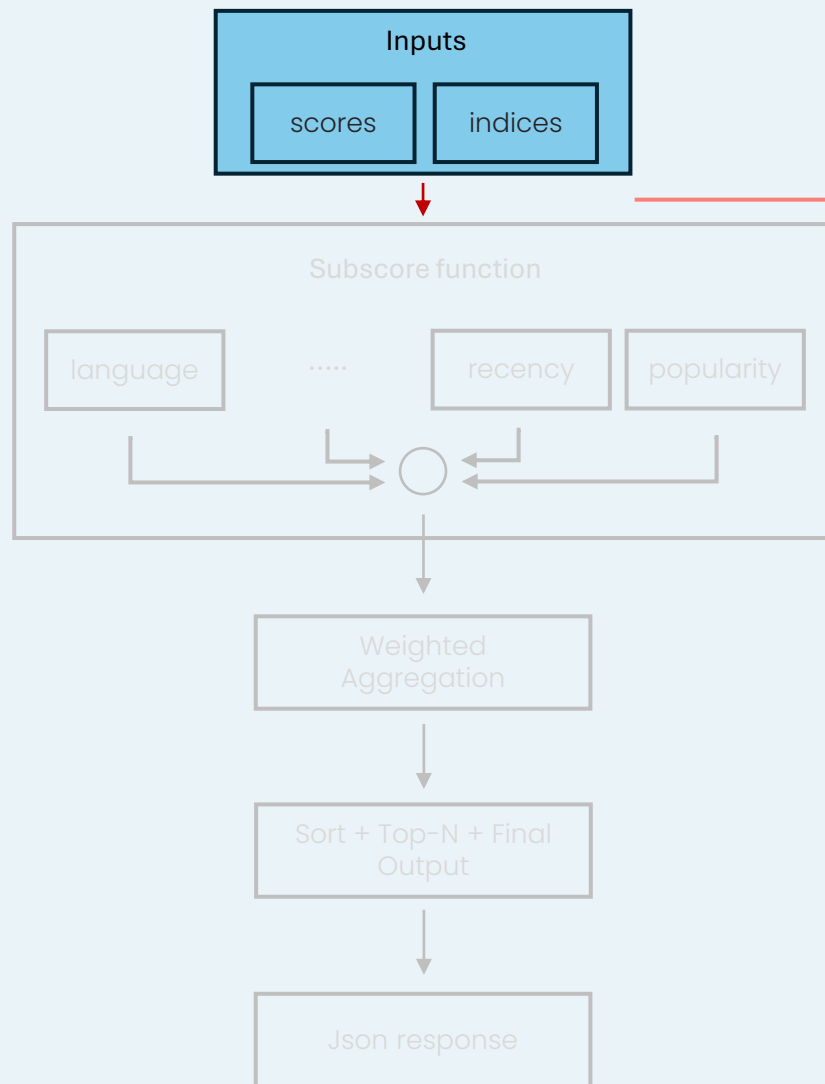
Overview



```
scores -> [[0.991958 0.90618014 0.8808831 0.8787683 0.8786377 0.87398016
0.8577175 0.8541027 0.8468044 0.84393305 0.84289694 0.8399202
0.83722746 0.830644 0.82787627 0.8273529 0.82603955 0.8251338
0.8238946 0.8144704 0.81317806 0.81250346 0.8115001 0.80699027
0.79952943 0.79905176 0.7947829 0.79342234 0.7903886 0.7769083 ]
[0.96994966 0.9090964 0.9057696 0.9006964 0.8729689 0.8631804
0.8566084 0.85157895 0.8515715 0.8505182 0.8484962 0.84781826
0.8467041 0.8420427 0.8355931 0.8284064 0.81695354 0.80033326
0.793495 0.7910472 0.7855313 0.7844976 0.77936697 0.77754533
0.7763233 0.7762666 0.7715549 0.76181847 0.76118565 0.7558825 ]
[0.914314 0.9138078 0.90648955 0.9038511 0.9035522 0.9006585
0.8928431 0.8666352 0.86223924 0.85702205 0.85435927 0.8537438
0.8505054 0.84931517 0.84525776 0.8431046 0.8385591 0.83225965
0.82806206 0.82543594 0.821131 0.8152864 0.81420815 0.80876017
0.80735594 0.80000085 0.79682016 0.79644376 0.7904746 0.78952384]
[0.8820293 0.8762982 0.8635522 0.8632113 0.8617437 0.85991806
0.8592367 0.8521462 0.83182544 0.82940936 0.8234046 0.81882125
0.8164314 0.8125658 0.8109041 0.8102988 0.8094929 0.80714226
0.805785 0.8035424 0.800296 0.7932352 0.78877866 0.78805614
0.7869439 0.7789126 0.77490985 0.7742216 0.76786566 0.767448 ]
[0.9367956 0.8867005 0.8791922 0.8721591 0.87159765 0.866328
0.8644314 0.86207354 0.854147 0.8470825 0.84155536 0.84109956
0.8349133 0.82845306 0.8267503 0.82358754 0.8219552 0.8219016
0.8216283 0.82084084 0.8194456 0.81671 0.81300527 0.81182337
0.8108462 0.80841625 0.8032939 0.8021481 0.800935 0.7986978 ]]
indices -> [[30 32 41 38 34 52 46 50 35 39 57 47 37 31 55 58 44 54 36 43 33 40 45 56
17 42 51 53 15 49]
[32 37 30 52 34 58 43 35 56 39 47 38 54 41 33 36 45 50 46 51 40 53 26 44
48 31 15 17 57 19]
[30 52 35 38 41 32 34 46 50 37 39 45 31 40 33 43 47 36 58 53 44 51 57 54
48 56 55 15 13 17]
[34 46 30 35 32 31 41 52 38 39 54 58 57 43 45 36 47 50 33 37 15 56 53 44
40 42 51 19 14 18]
[34 41 32 35 39 30 52 43 15 10 14 33 53 38 36 57 31 45 18 6 54 11 58 37
56 47 46 19 44 1]]
```

HyDE Feed Recommendation - Subscore

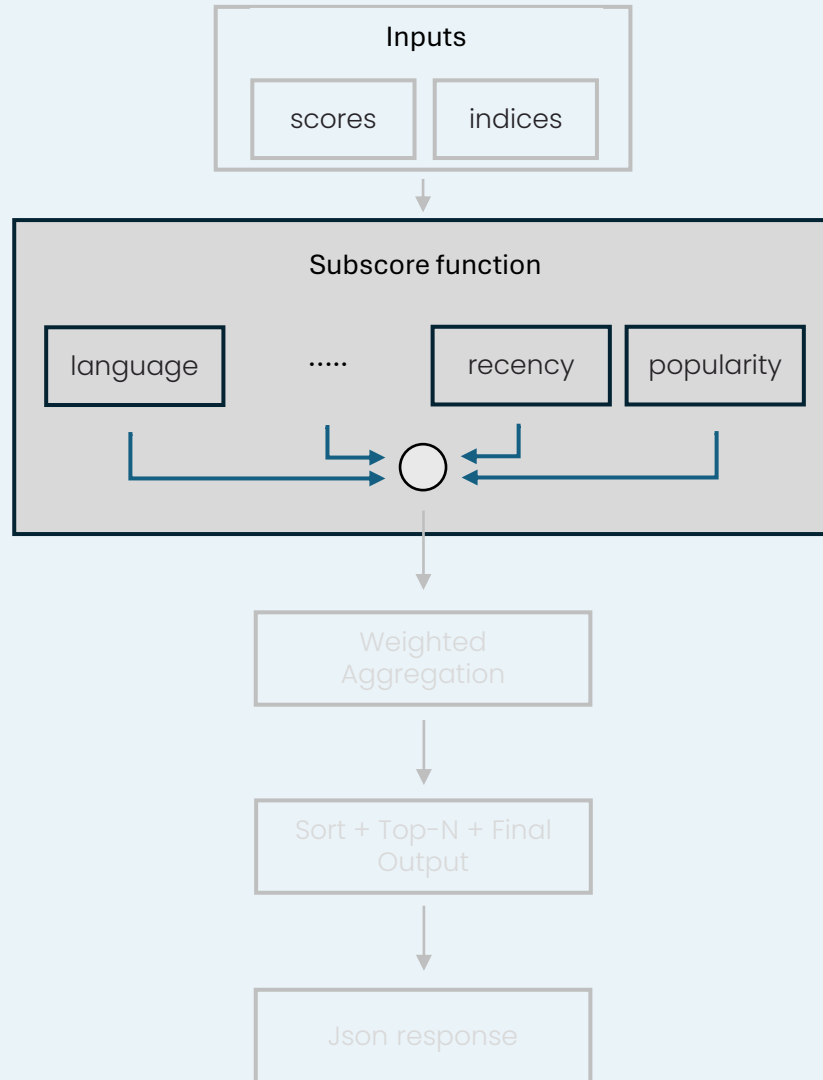
Overview



```
qi:0 -> wi:1.0
- ki : 0
- idx : 30 -> raw : 0.9919580221176147
- feed_id -> TH_F001
  - cand -> 0.9919580221176147
  - cand -> None
- ki : 1
- idx : 32 -> raw : 0.9061801433563232
- feed_id -> TH_F003
  - cand -> 0.9061801433563232
  - cand -> None
- ki : 2
- idx : 41 -> raw : 0.8808830976486206
- feed_id -> TH_F012
  - cand -> 0.8808830976486206
  - cand -> None
- ki : 3
- idx : 38 -> raw : 0.8787683248519897
- feed_id -> TH_F009
  - cand -> 0.8787683248519897
  - cand -> None
- ki : 4
- idx : 34 -> raw : 0.8786376714706421
- feed_id -> TH_F005
  - cand -> 0.8786376714706421
  - cand -> None
- ki : 5
- idx : 52 -> raw : 0.8739801645278931
- feed_id -> TH_F023
  - cand -> 0.8739801645278931
  - cand -> None
```

HyDE Feed Recommendation - Subscore

Overview

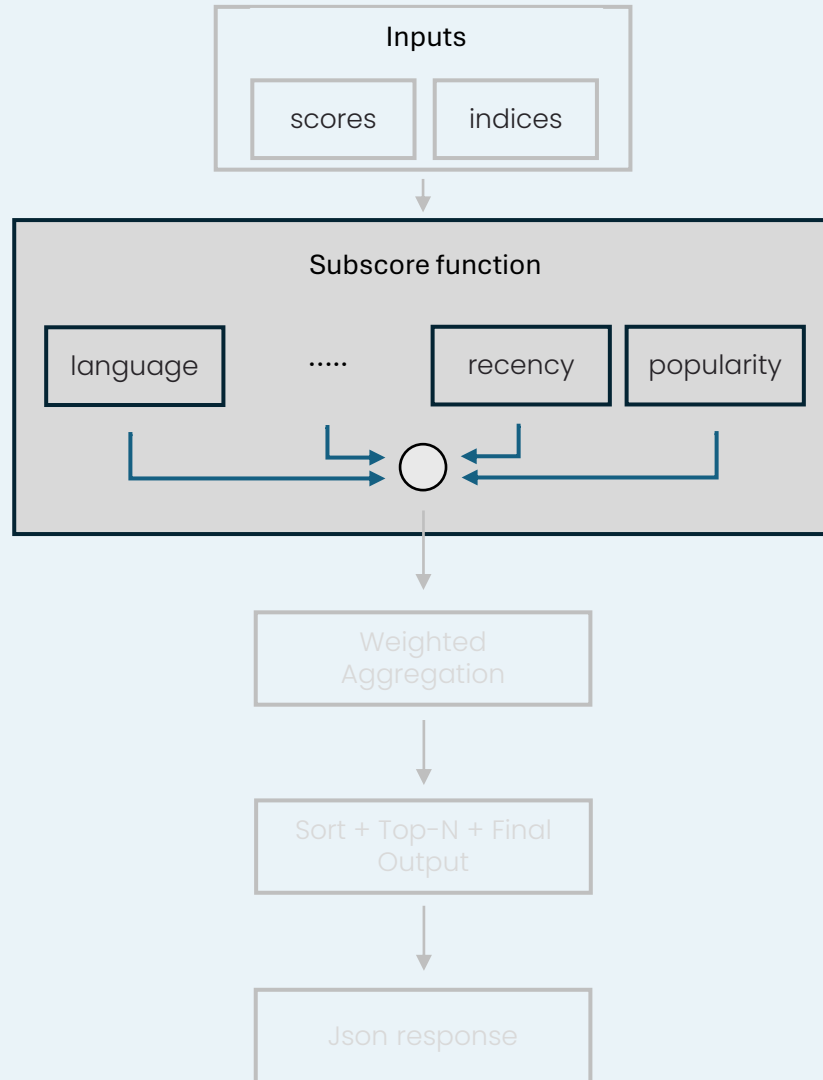


Mapping retrieval outputs to real feed items

- We got internal vector ids (indices), NOT business ids
- We convert ``index_id`` -> ``feed_id`` via aligned metadata array
- Then join ``feed_id`` -> full feed metadata from `feeds_meta.jsonl`
- Results candidates become “real feed items” with fields like title, language created_at, views

HyDE Feed Recommendation - Subscore

Overview



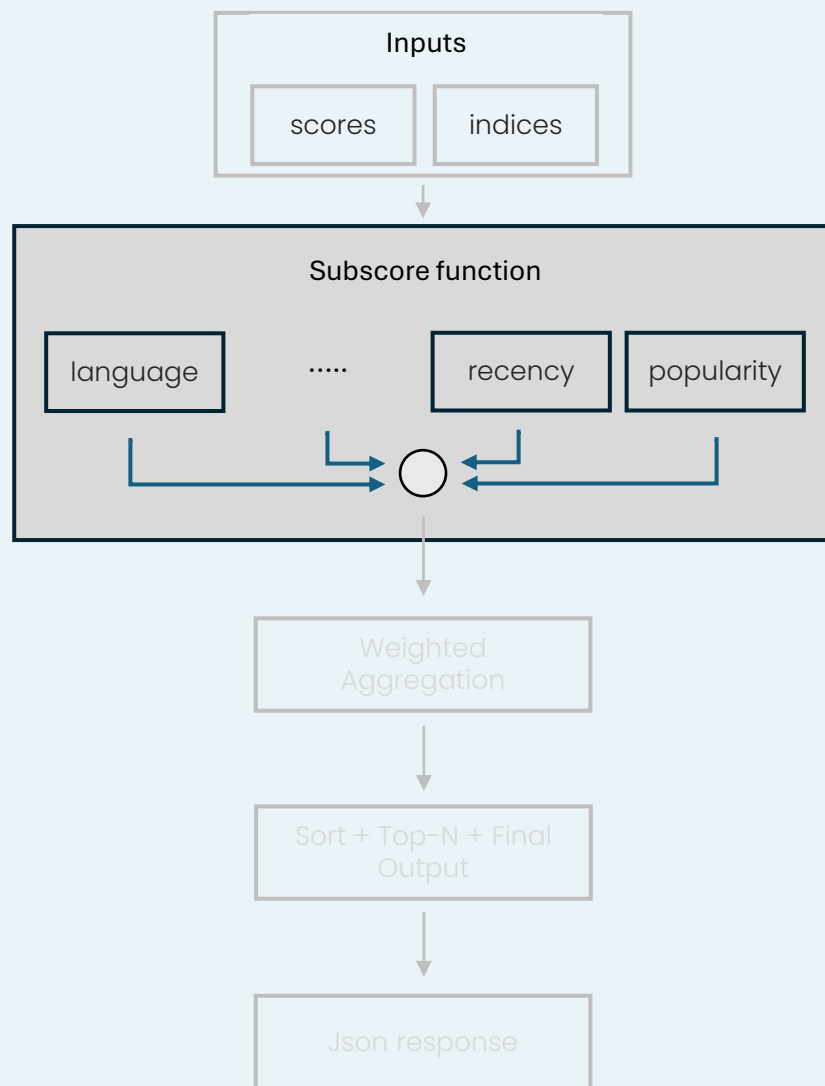
Deterministic Subscore

- We compute 3 interpretable signals per feed
 1. Language_match : exact match
 2. Recency : exponential decay using half-life days
 3. Popularity : log-normalized engagement (views/likes/etc.)
- All score are bounded in $[0, 1]$
- Subscore are deterministic (same inputs -> same output)

```
subscores = {  
  "language_match": score_language_match(fmeta, user_lang=user_lang),  
  "recency": score_recency(fmeta, now_utc=now_utc, half_life_days=recency_half_life_days),  
  "popularity": score_popularity(fmeta)  
}
```

HyDE Feed Recommendation - Subscore

Overview



Deterministic Subscore

- We compute 3 interpretable signals per feed

1. Language_match : exact match
2. Recency : exponential decay using half-life days
3. Popularity : log-normalized engagement (views/likes/etc.)

- Read feed language from : `feed_meta["lang"]`

- If equal user language -> 1.0 else ->0.0

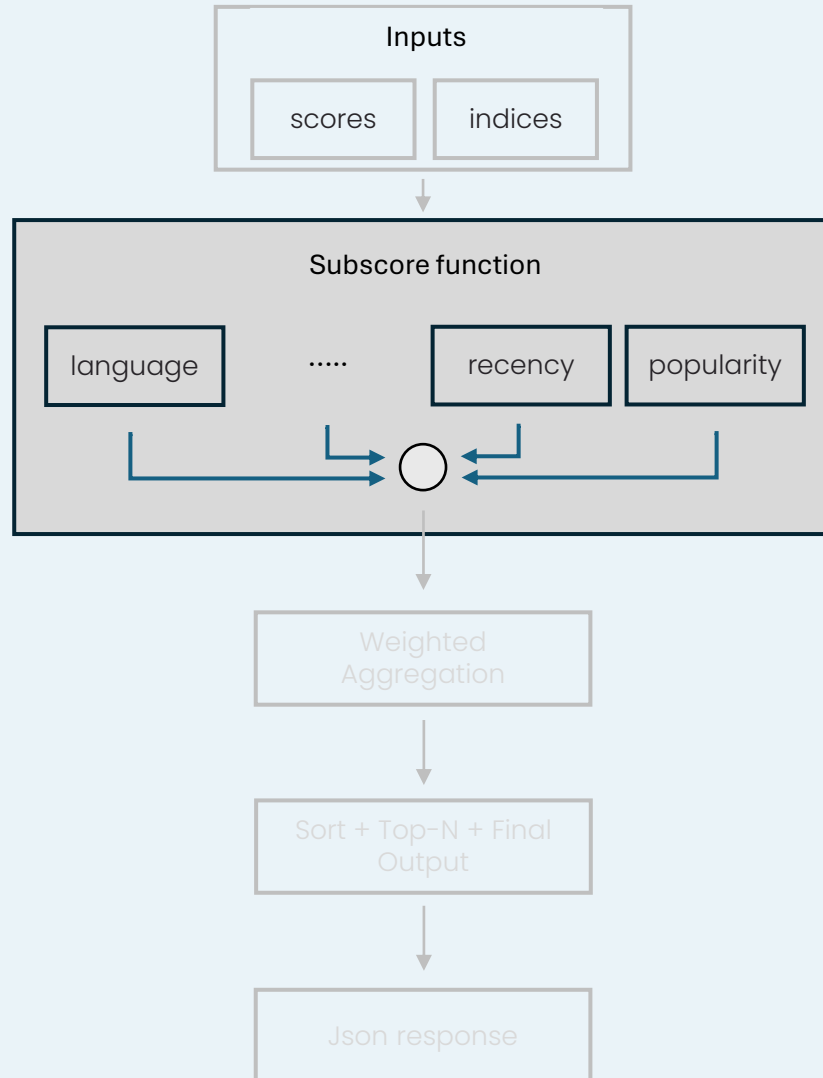
- It's a hard match (not similarity, just exact match)

- If user prefers "th" , Thai content gets a boost

```
# -----  
# Language match score  
# -----  
def score_language_match(feed_meta: Dict[str,Any],*,user_lang: Optional[str]) -> float:  
    """Score exact language match between user and feed."""  
    if not user_lang:  
        return 0.0  
    feed_lang = feed_meta.get("lang") or feed_meta.get("language")  
    if isinstance(feed_lang,str) and feed_lang.strip().lower() == user_lang.strip().lower():  
        return 1.0  
    return 0.0
```

HyDE Feed Recommendation - Subscore

Overview



Deterministic Subscore

- We compute 3 interpretable signals per feed

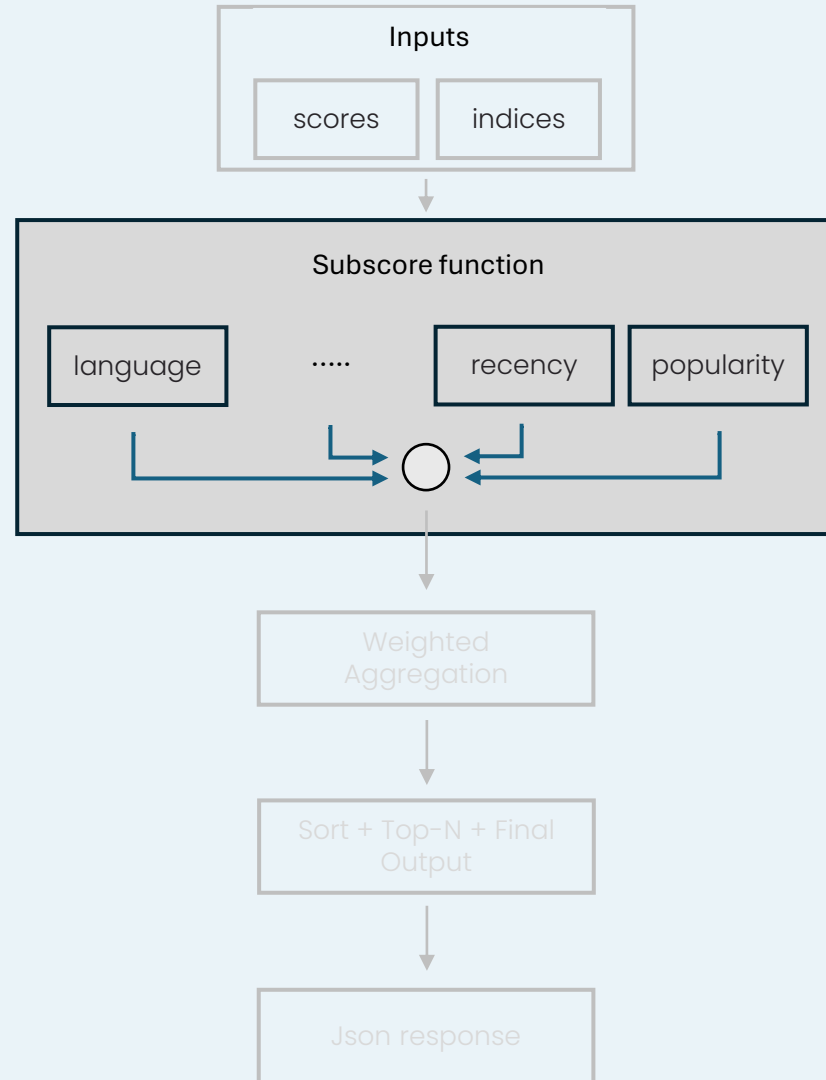
1. Language_match : exact match
2. Recency : exponential decay using half-life days
3. Popularity : log-normalized engagement (views/likes/etc.)

$$\text{recency} = 2^{-\frac{\text{age_days}}{\text{half_life_days}}}$$

```
# -----  
# Recency score  
# -----  
def score_recency(  
    feed_meta: Dict[str, Any],  
    *,  
    now_utc: datetime,  
    half_life_days: float = 30.0  
)-> float:  
    """Compute an exponential time-decay recency score in [0,1]  
    score = 2 ^ (-age_days / half_life_days)  
    """  
    for key in ("published_at", "created_at", "timestamp", "date", "ts"):  
        dt = parse_ts_any(feed_meta.get(key))  
        if dt is None:  
            continue  
        age_days = (now_utc - dt).total_seconds() / 86400.0  
        if age_days < 0:  
            age_days = 0.0  
        if half_life_days <= 0:  
            return 0.0  
        return float(2.0**(-(age_days/half_life_days)))  
    return 0.0
```

HyDE Feed Recommendation - Subscore

Overview



Deterministic Subscore

- We compute 3 interpretable signals per feed

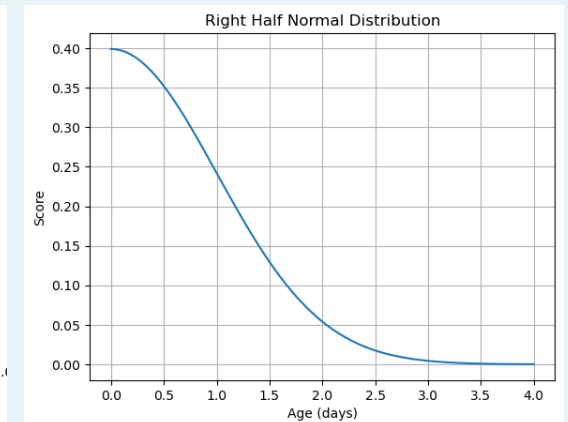
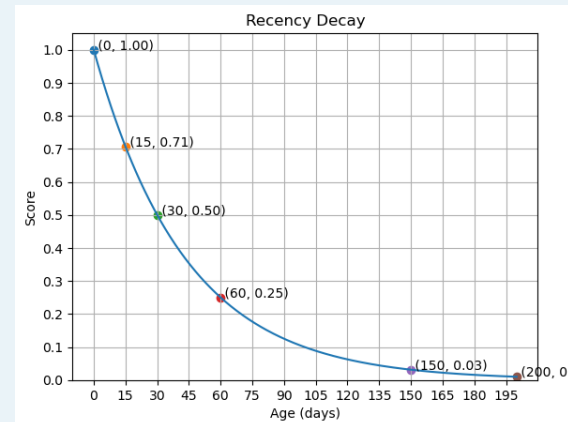
1. `Language_match` : exact match
2. `Recency` : exponential decay using half-life days
3. `Popularity` : log-normalized engagement (views/likes/etc.)

$$\text{recency} = 2^{-\frac{\text{age_days}}{\text{half_life_days}}}$$

- This is a “freshness score”

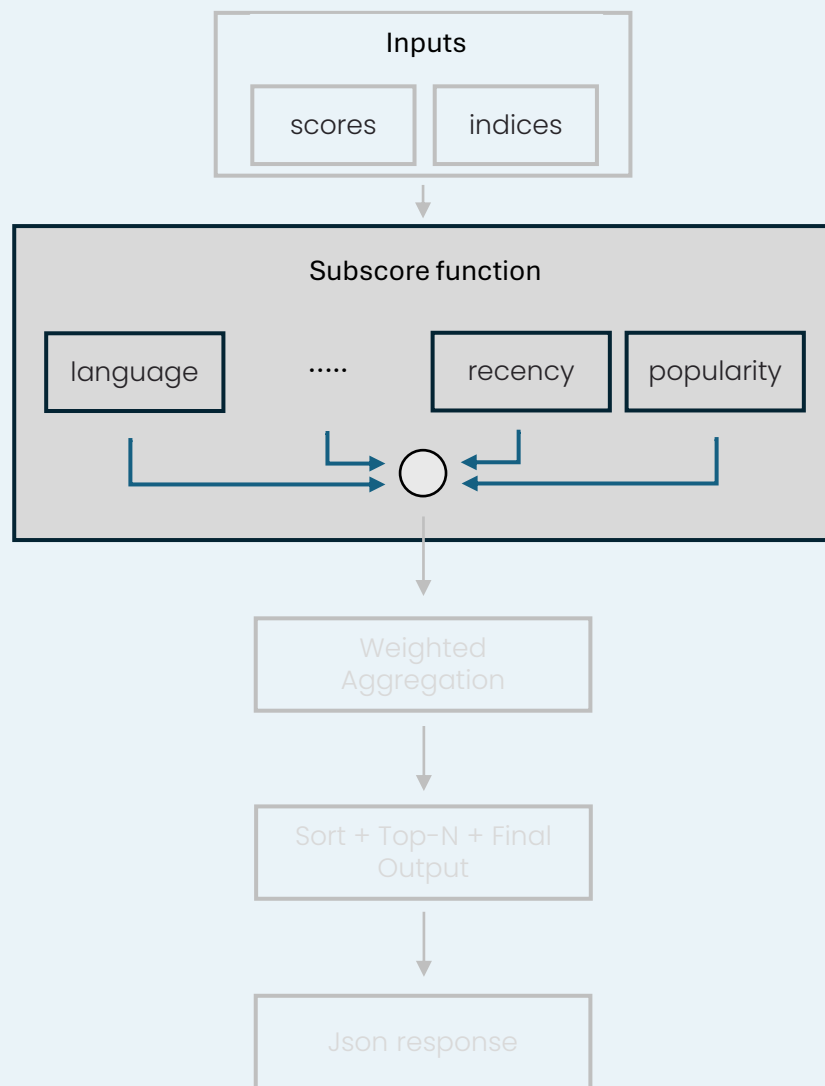
- age = 0 days -> recency = 1.0 ($2^{*(-0/30)}$ -> 1.0)
- age = 15 days -> recency = 0.707 ($2^{*(-15/30)}$ -> 0.707)
- age = 30 days -> recency = 0.5 ($2^{*(-30/30)}$ -> 0.5)
- age = 60 days -> recency = 0.25 ($2^{*(-60/30)}$ -> 0.25)

- The half - life is configurable (recency_half_life_days from YAML)



HyDE Feed Recommendation - Subscore

Overview



Deterministic Subscore

- We compute 3 interpretable signals per feed

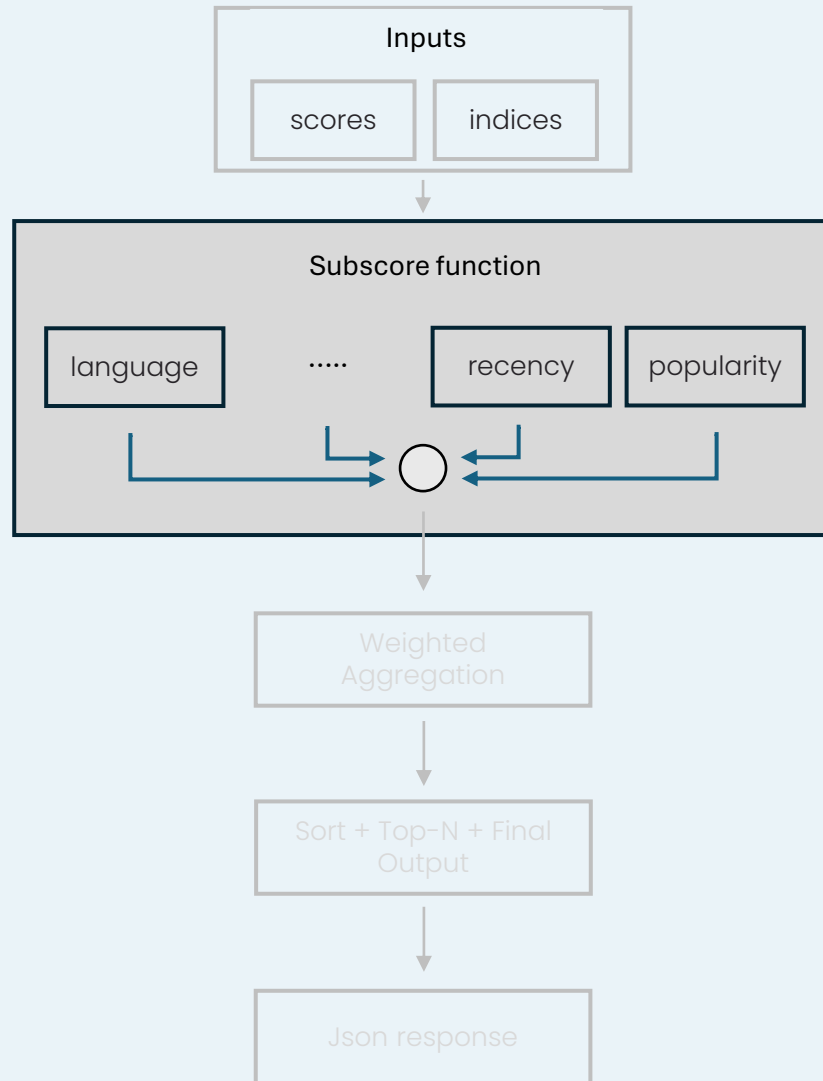
1. Language_match : exact match
2. Recency : exponential decay using half-life days
3. Popularity : log-normalized engagement (views/likes/etc.)

$$popularity = \min\left(1, \frac{\log(1 + v)}{\log(1 + 1,000,000)}\right)$$

```
# -----  
# Popularity score  
# -----  
def score_popularity(feed_meta:Dict[str,Any]) -> float:  
    """Compute a bounded popularity score in [0,1] using log scaling  
    score = log1p(v) / log1p(1_000_000)  
    """  
    for key in ("popularity","views","likes","clicks","impressions"):  
        v = feed_meta.get(key)  
        if isinstance(v, (int,float)) and v>0:  
            return float(  
                min(  
                    1.0,  
                    np.log1p(float(v)) / np.log1p(1_000_000.0),  
                )  
            )  
    return 0.0
```


HyDE Feed Recommendation - Subscore

Overview



Deterministic Subscore

- We compute 3 interpretable signals per feed

1. Language_match : exact match
2. Recency : exponential decay using half-life days
3. Popularity : log-normalized engagement (views/likes/etc.)

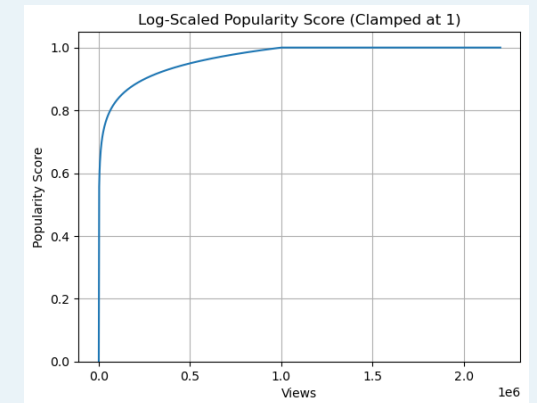
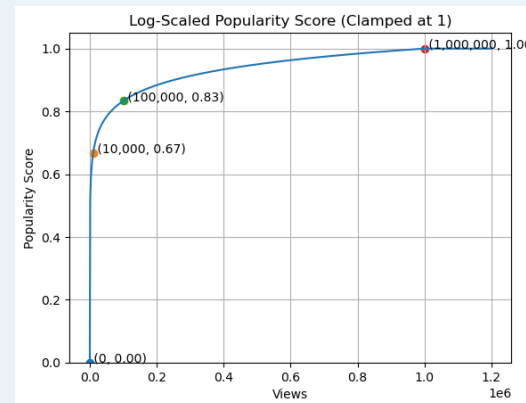
$$popularity = \min\left(1, \frac{\log(1 + v)}{\log(1 + 1,000,000)}\right)$$

- This is a “popularity score”

- view = 0 → popularity = 0.0 ($\log(1) / 13.8155 \rightarrow 0$)
- view = 10,000 → popularity = 0.67 ($\log(10,001) / 13.8155 \rightarrow 0.67$)
- view = 100,000 → popularity = 0.83 ($\log(100,001) / 13.8155 \rightarrow 0.83$)
- view = 1,000,000 → popularity = 1.0 ($\log(1,000,001) / 13.8155 \rightarrow 1.0$)

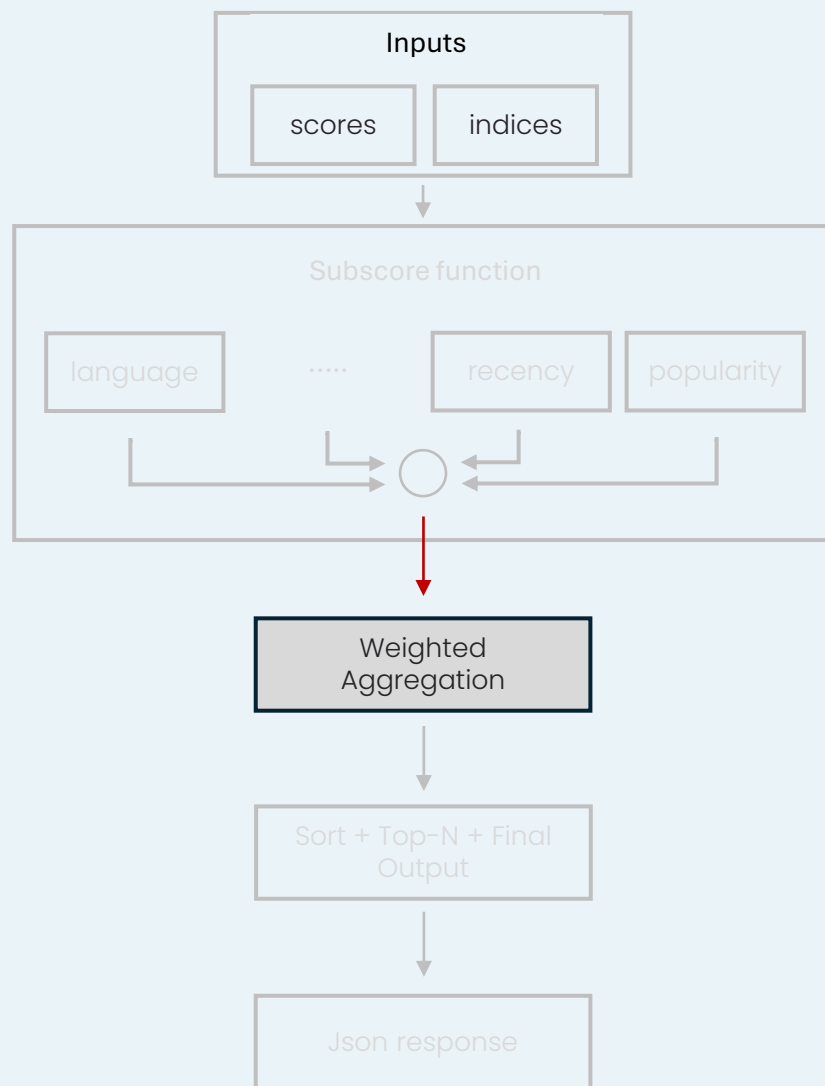
- Popular feeds get boost But they do NOT overpower semantic similarity

- We clamp to 1 (Prevents runaway values)



HyDE Feed Recommendation - Subscore

Overview



Weighted Aggregation

- Subscore + vector score are combined into one final_score

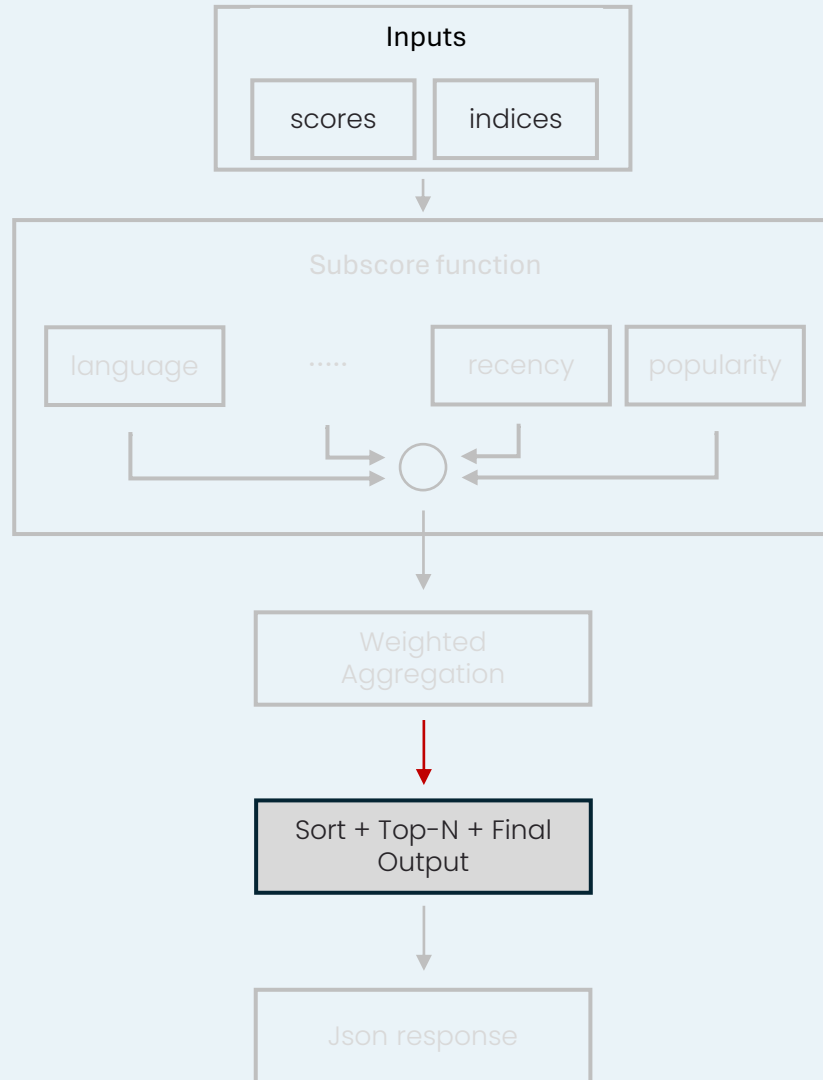
$$final_score = \sum_i weight_i * clamp(value)$$

- YAML controls weights (easy to tune without touching code)
 - which features participate
 - their weights
 - whether to clamp inputs
 - whether to renormalize weights (sum to 1)

```
{'feed_id': 'TH_F001',
 'vector_score': 0.9919580221176147,
 'weight':{
   "language_match":0.2,
   "recency":0.4,
   "popularity":0.4
 },
 'subscores': {'language_match': 1.0,
 'recency': 8.568584468944074e-05,
 'popularity': 0.8182601703827922},
 'feed_header': 'แนวทางการทำพอร์ต Data Analyst ด้วยโปรแกรม Python และ SQL',
 'debug': {'best_query_id': 'Q1',
 'best_query_intent': 'history_aligned',
 'best_raw_score': 0.9919580221176147,
 'best_weight': 1.0}
}
```

HyDE Feed Recommendation - Subscore

Overview

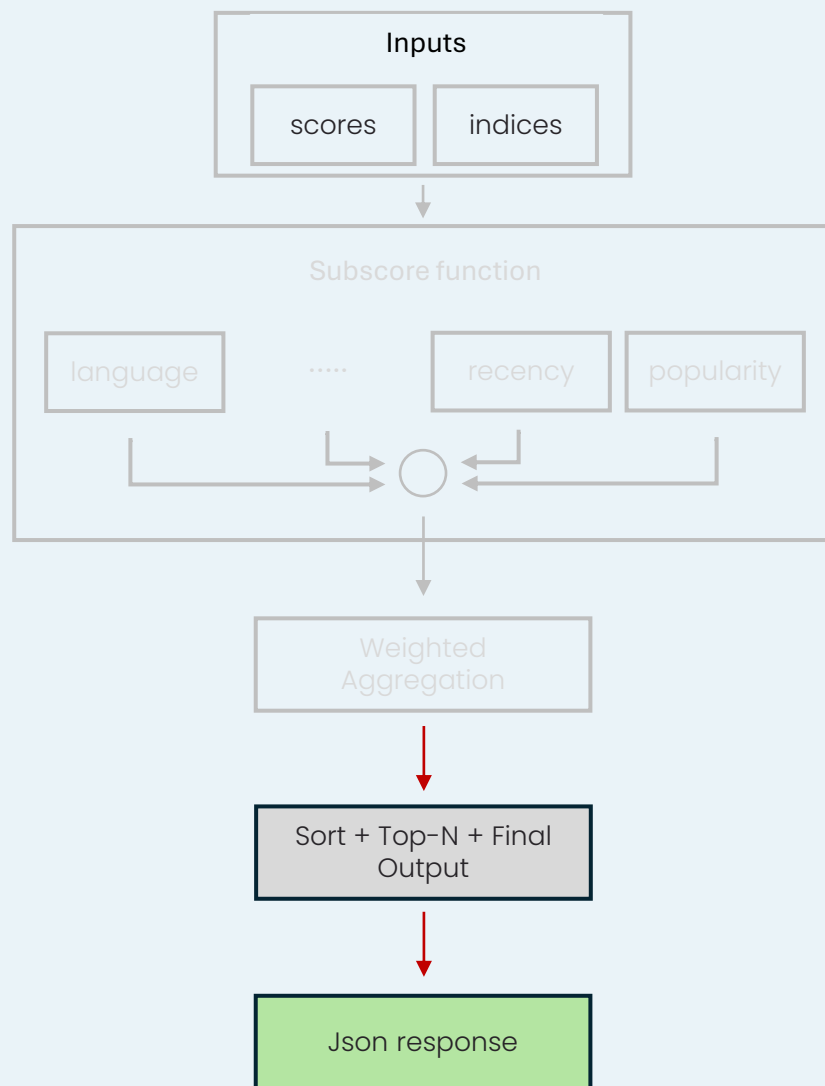


Sort + Top-N + Final output

- Sort candidates by ranking score key
- Return
 - student_id
 - meta configuration for debugging / reproducibility
 - candidate list (previews/top-N)
- Candidate includes both
 - raw retrieval score (vector_score)
 - interpretable subscores (why it ranked)

HyDE Feed Recommendation - Subscore

Overview



Sort + Top-N + Final output

```
=== Preview (top 5) ===
- TH_F001 | แนวทางทำพอร์ค Data Analyst ด้วยโปรแกรม Python และ SQL | score=0.5952
- TH_F003 | เทคนิคเตรียมสัมภาษณ์งาน Data Analyst (SQL + Python) | score=0.5820
- TH_F023 | Data Analyst Intern: ตัวอย่างโจทย์ Take-home และวิธีทำให้อัปเกรด | score=0.5483
- TH_F008 | รวมโจทย์ SQL ผูกสัมภาษณ์ระดับฝึกงาน (พร้อมเฉลยแนวคิด) | score=0.5455
- TH_F006 | พื้นฐานสถิติที่ใช้อยู่ในงาน Data Analyst (พร้อมตัวอย่าง) | score=0.5439
{
  "student_id": "stu_p001",
  "num_candidates": 40,
  "meta": {
    "top_k_per_query": 30,
    "max_candidates": 200,
    ...
  },
  "score_aggregation": {
    "enabled": true,
    "mode": "linear",
    "weights": {
      "vector_score": 0.6,
      "language_match": 0.3,
      "recency": 0.05,
      "popularity": 0.05
    },
    "clamp_inputs": true,
    ...
  },
  "candidates_preview": [
    {
      "feed_id": "TH_F001",
      "vector_score": 0.9919580221176147,
      "subscores": {
        "language_match": 1.0,
        "recency": 8.568584468944074e-05,
        "popularity": 0.8182601703827922
      },
      "feed_header": "แนวทางทำพอร์ค Data Analyst ด้วยโปรแกรม Python และ SQL",
      "debug": {
        "best_query_id": "Q1",
        "best_query_intent": "history_aligned",
        "best_raw_score": 0.9919580221176147,
        "best_weight": 1.0
      },
      "final_score": 0.5951748132705689
    },
    ...
  ]
}
```

Sprint review #10

HyDE feed recommendation

API Spec Link :

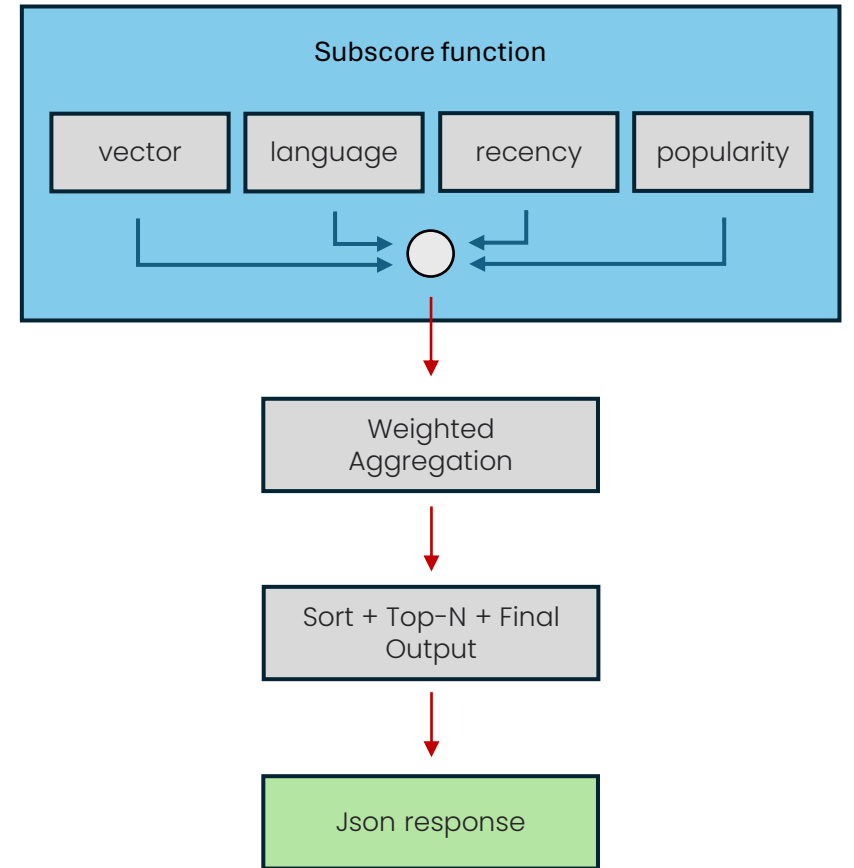
Orchestrator :

API core evaluation link : <https://hyderecomment-service-v1-1-0-du7yhkyaqg-as.a.run.app/docs#/>

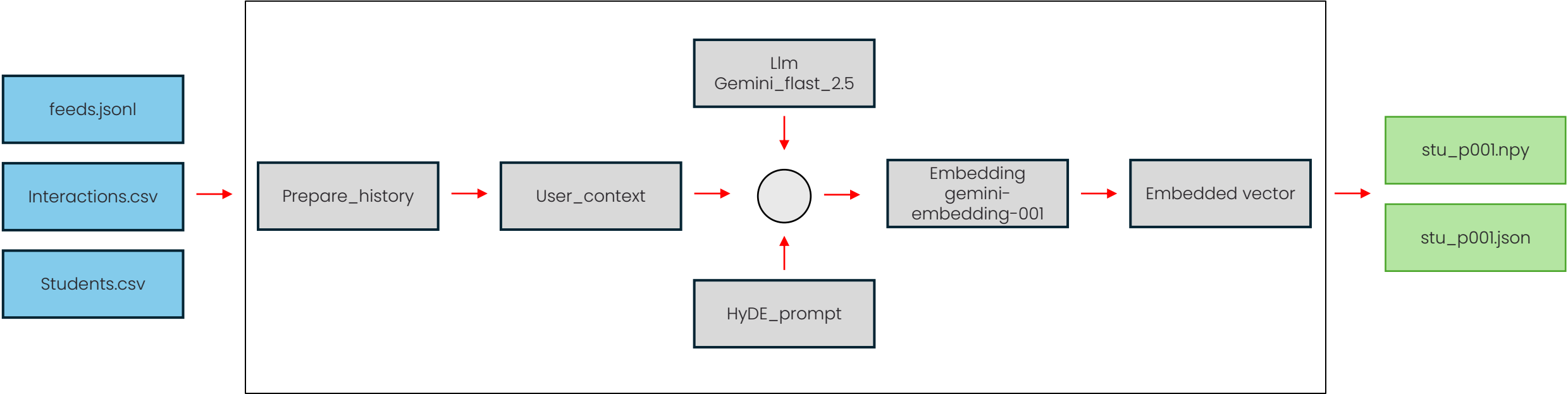
Github : <https://github.com/TunKedsaro/Hyde>

QA : <https://humanintelligence.atlassian.net/wiki/spaces/CPM/pages/134611027/QA+Test+Plan+CV+Evaluation+Pipeline>

HyDE Feed Recommendation - System Overview



HyDE Feed Recommendation - System Overview



Legend



↔ Process

HyDE Feed Recommendation - System Overview

feeds.jsonl

Interactions.csv

Students.csv



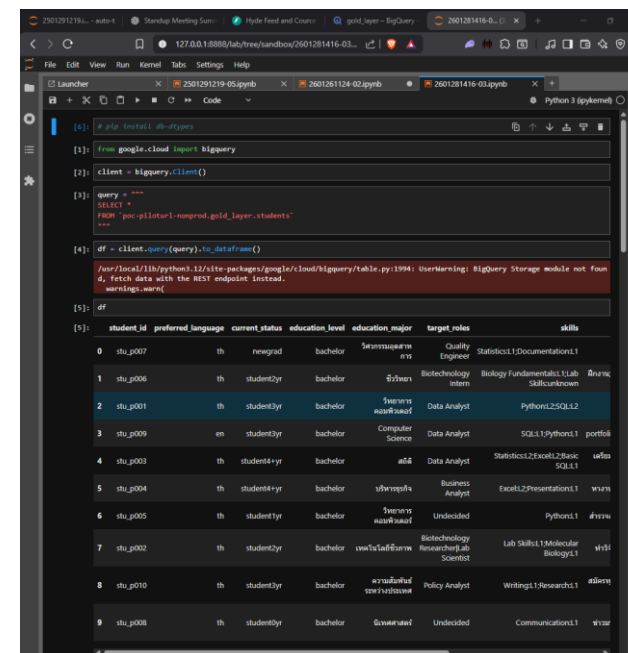
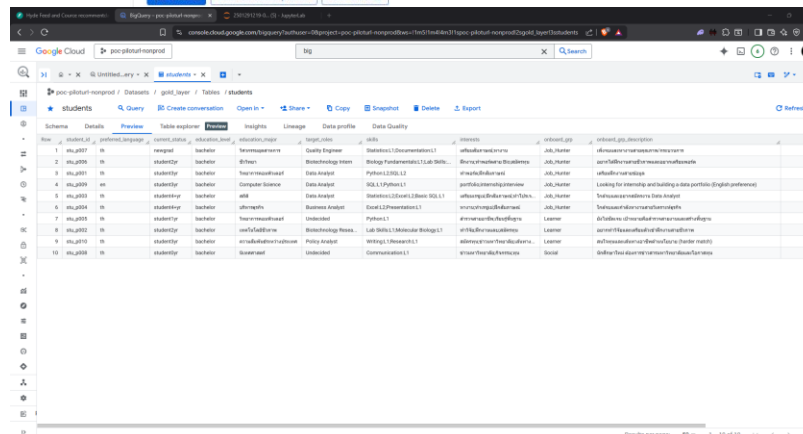
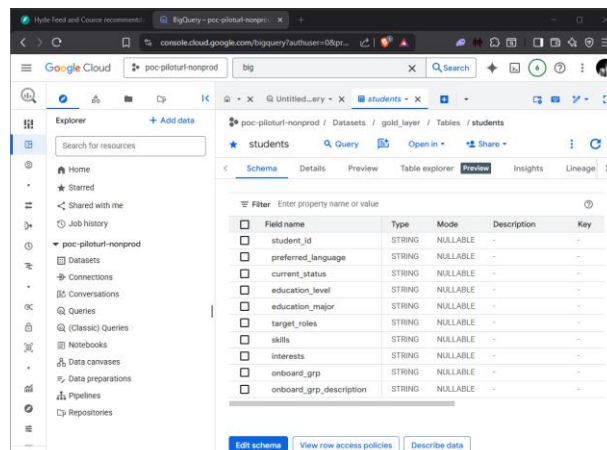
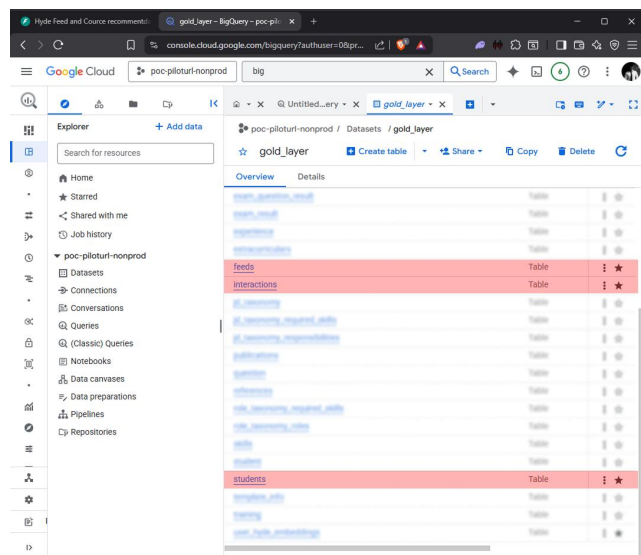
Google
Big Query



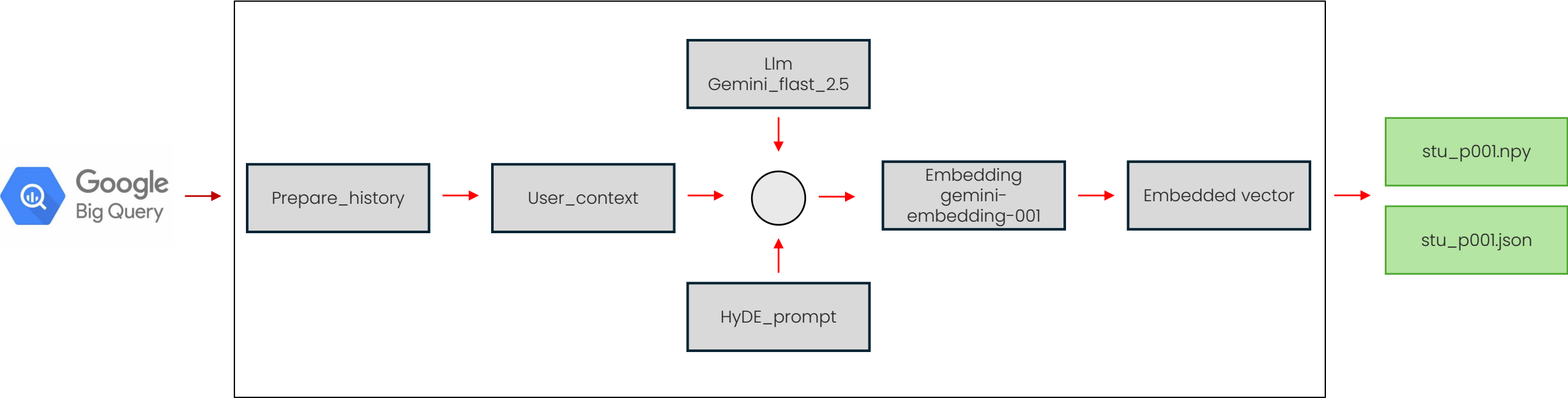
The screenshot shows the Google Cloud BigQuery console interface. The left sidebar displays the Explorer view with a search bar and a list of resources. The main panel shows the 'gold_layer' dataset with a list of tables. The tables 'feeds', 'interactions', and 'students' are highlighted in red.

Table Name	Type	Star
exam_question_result	Table	
exam_result	Table	
experience	Table	
extra-curriculars	Table	
feeds	Table	★
interactions	Table	★
is_recruiting	Table	
is_recruiting_required_schls	Table	
is_recruiting_responsibilities	Table	
publications	Table	
questions	Table	
references	Table	
role_recruiting_required_schls	Table	
role_recruiting_roles	Table	
schls	Table	
student	Table	
students	Table	★
templates_schls	Table	
training	Table	
user_hyde_embeddings	Table	

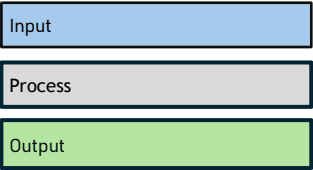
HyDE Feed Recommendation - System Overview



HyDE Feed Recommendation - System Overview



Legend

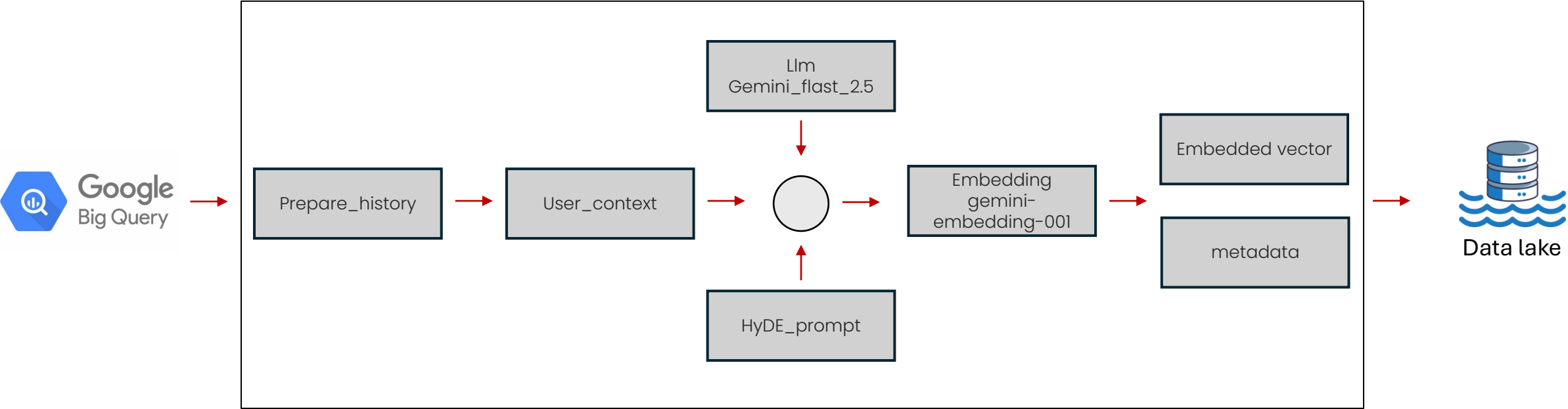


Process

HyDE Feed Recommendation - System Overview



HyDE Feed Recommendation - System Overview



Legend

Input

Process

Output

↔

 Process

HyDE Feed Recommendation - System Overview

The screenshot displays the Google Cloud Storage console interface. The left sidebar shows the navigation menu with 'Cloud Storage' selected. The main content area shows the 'hyde-datalake-feeds' bucket details, including its location (asia-southeast1), storage class (Standard), and public access (Not public). The 'Objects' tab is active, showing a folder browser on the left and a list of objects on the right. The folder browser shows a hierarchy: 'hyde-datalake-feeds' > 'stu_p010' > 'embedding'. The object list shows five files, each 3.2 KB in size, with names like 'embedding01.npy' through 'embedding05.npy'. A notification at the bottom states 'Public URL copied to clipboard'.

Google Cloud console showing the 'hyde-datalake-feeds' bucket details and the 'embedding' folder contents.

Bucket details:

- Location: asia-southeast1 (Singapore)
- Storage class: Standard
- Public access: Not public
- Protection: Soft Delete

Folder browser:

- hyde-datalake-feeds
 - stu_p001/
 - stu_p002/
 - stu_p003/
 - stu_p004/
 - stu_p005/
 - stu_p006/
 - stu_p007/
 - stu_p008/
 - stu_p009/
 - stu_p010/
 - embedding/
 - metadata/

Object list:

Name	Size	Type	Created	Storage class	Last modified	Public access
embedding01.npy	3.2 KB	application/octet-stream	Jan 30, 2026, 8:40:25 PM	Standard	Jan 30, 2026, 8:40:25 PM	Not public
embedding02.npy	3.2 KB	application/octet-stream	Jan 30, 2026, 8:40:25 PM	Standard	Jan 30, 2026, 8:40:25 PM	Not public
embedding03.npy	3.2 KB	application/octet-stream	Jan 30, 2026, 8:40:25 PM	Standard	Jan 30, 2026, 8:40:25 PM	Not public
embedding04.npy	3.2 KB	application/octet-stream	Jan 30, 2026, 8:40:26 PM	Standard	Jan 30, 2026, 8:40:26 PM	Not public
embedding05.npy	3.2 KB	application/octet-stream	Jan 30, 2026, 8:40:26 PM	Standard	Jan 30, 2026, 8:40:26 PM	Not public

Public URL copied to clipboard

HyDE Feed Recommendation - System Overview

```
hyde-datalake-feeds/  
├─ stu_p001/  
│   ├── metadata/  
│   │   └─ metadata.json  
│   └─ embedding/  
│       ├── embedding01.npy  
│       ├── embedding02.npy  
│       ├── embedding03.npy  
│       ├── embedding04.npy  
│       └─ embedding05.npy  
├─ stu_p002/  
├─ ...  
└─ stu_p010/
```

HyDE Feed Recommendation - System Overview

```
hyde-datalake-feeds/  
├─ stu_p001/  
│   ├── metadata/  
│   │   └─ metadata.json  
│   ├── embedding/  
│   │   ├── embedding01.npy  
│   │   ├── embedding02.npy  
│   │   ├── embedding03.npy  
│   │   ├── embedding04.npy  
│   │   └─ embedding05.npy  
├─ stu_p002/  
└─ ...  
└─ stu_p010/
```



```
1 gcs.read_json(  
2     blob_path = "stu_p001/metadata/metadata.json"  
3 )
```

Python

```
{'student_id': 'stu_p001',  
 'current_status': 'student3yr',  
 'education_level': 'bachelor',  
 'education_major': 'วิทยาการคอมพิวเตอร์',  
 'target_roles': 'Data Analyst',  
 'timezone': 'UTC',  
 'model_name': 'gemini-2.5-flash',  
 'max_output_tokens': 2048,  
 'feed_text_max_chars': 240,  
 'temperature': 0.2}
```

HyDE Feed Recommendation - System Overview

```
hyde-datalake-feeds/  
├─ stu_p001/  
│   ├── metadata/  
│   │   └─ metadata.json  
│   └─ embedding/  
│       ├── embedding01.npy  
│       ├── embedding02.npy  
│       ├── embedding03.npy  
│       ├── embedding04.npy  
│       └─ embedding05.npy  
├─ stu_p002/  
├─ ...  
└─ stu_p010/
```

```
1 gcs.read_npy(  
2     blob_path = "stu_p001/embedding/embedding01.npy"  
3 )
```

Python

```
array([ 1.60014424e-02, -6.24772394e-03,  1.08490810e-02, -1.11342959e-01,  
       -1.91002712e-02,  1.55251492e-02, -1.24348572e-03, -1.42529728e-02,  
       -1.71437729e-02, -6.21889858e-03, -5.35096452e-02,  3.33515299e-03,  
        5.24768746e-03, -6.04540855e-03,  1.91499725e-01,  1.27674532e-04,  
       -1.70064252e-02,  7.98255391e-03,  9.49927326e-03, -3.15444618e-02,  
        1.70750413e-02,  2.63916198e-02, -4.38489318e-02, -6.69290451e-03,  
        6.13231864e-03, -2.42059473e-02,  7.74292182e-03,  5.33192977e-02,  
        3.45389880e-02, -1.75931584e-02,  6.41446933e-03,  3.40056792e-02,  
        5.12950048e-02, -5.91142289e-03, -1.44502996e-02, -1.69220176e-02,  
        6.64372789e-03, -5.20436047e-03, -5.13289822e-04, -1.93110611e-02,  
       -4.83487593e-03,  4.01458889e-02, -2.57270448e-02, -1.33291562e-03,  
       -5.75538538e-03, -8.56110855e-05,  9.59179085e-03, -8.27102289e-02,  
       -3.05592245e-03, -6.98679313e-03,  2.21752971e-02,  1.95462313e-02,  
       -5.18478602e-02, -3.40254188e-01,  2.25967285e-03, -7.62488274e-03,  
       -1.40009578e-02, -1.21486178e-02, -5.12464531e-02, -8.00063927e-03,  
       -2.30011884e-02,  5.41142039e-02, -7.29405433e-02, -1.20696076e-03,  
        3.81701067e-02, -8.49270355e-03,  5.78285195e-02, -2.45383084e-02,  
       -3.00421212e-02, -5.25304349e-03,  3.47595513e-02, -5.34191867e-03,  
       -4.09709178e-02, -4.74740975e-02, -3.30488905e-02, -5.22667915e-02,  
       -4.33363812e-03,  7.27543328e-03,  1.46426624e-02,  3.49784605e-02,
```


HyDE Feed Recommendation - System Overview

Video

HyDE Feed Recommendation

Pipeline Execution Time Breakdown per Student (seconds)

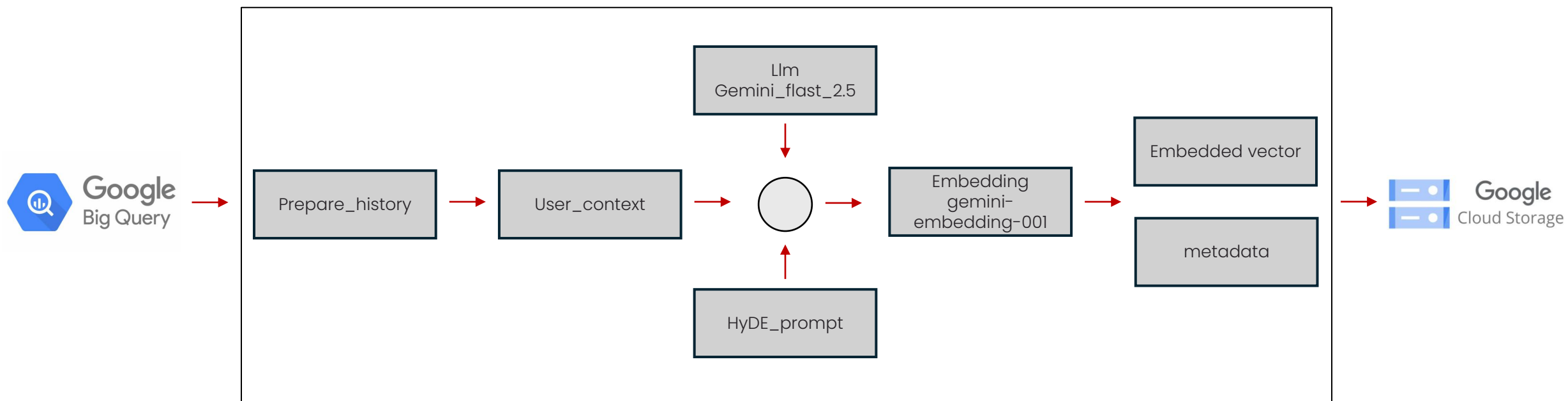
	Stu_p001	Stu_p002	Stu_p003	Stu_p004	Stu_p005	Stu_p006	Stu_p007	Stu_p008	Stu_p009	Stu_p010	average
query_sec	0.00101	0.00065	0.00058	0.00064	0.00067	0.00354	0.00077	0.00056	0.00056	0.00066	0.00096
history_sec	0.01339	0.01010	0.00979	0.00971	0.01111	0.01051	0.00888	0.00944	0.00959	0.01291	0.01054
embed_sec	1.81636	1.77266	2.03714	1.69645	1.72026	1.79597	1.75972	1.76440	1.72077	1.75303	1.78368
metadata.json	0.18499	0.12241	0.10613	0.11913	0.10847	0.11897	0.12841	0.11524	0.11533	0.12951	0.12486
embedding0n.npy	1.23484	1.28129	1.28272	1.16481	1.15355	1.12040	1.15927	1.15244	1.21216	1.12801	1.18895
total_sec	3.25059	3.18711	3.43636	2.99074	2.99306	3.04939	3.05705	3.04208	3.05841	3.02412	3.10898

HyDE Feed Recommendation

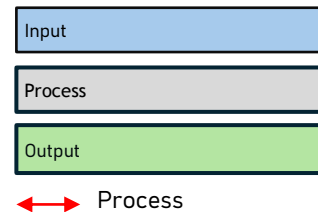
Storage retrieval from GCS (seconds)

	Stu_p001	Stu_p002	Stu_p003	Stu_p004	Stu_p005	Stu_p006	Stu_p007	Stu_p008	Stu_p009	Stu_p010	average
metadata.json	0.06906	0.0565	0.09909	0.06359	0.07367	0.06569	0.07023	0.0673	0.07545	0.07149	0.07514
embedding0n.npy	0.07413	0.06238	0.06575	0.07541	0.05905	0.06792	0.06209	0.06646	0.06743	0.06615	0.06786
total_sec	0.14319	0.11888	0.16484	0.139	0.13272	0.13361	0.13232	0.13376	0.14288	0.13764	0.143

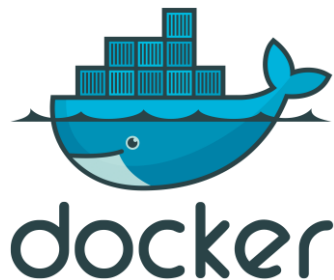
HyDE Feed Recommendation - System Overview



Legend



HyDE Feed Recommendation - System Architecture & Technology Stack



Gemini



Google Cloud Storage



Google
Big Query



Cloud Run



HyDE Feed Recommendation - System Architecture & Technology Stack

GET /health/gemini Health Gemini

GET /health/bigquery Health Bq

API:02 Bq -> project -> FastAPI

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://hyderecommend-service-v1-1-0-du7yhkyaqg-as.a.run.app/health/bigquery' \
  -H 'accept: application/json'
```

Request URL

```
https://hyderecommend-service-v1-1-0-du7yhkyaqg-as.a.run.app/health/bigquery
```

Server response

Code Details

200

Response body

```
{
  "status": "ok",
  "body": [
    {
      "student_id": "stu_p001",
      "preferred_language": "th",
      "current_status": "student3yr",
      "education_level": "bachelor",
      "education_major": "วิทยาการคอมพิวเตอร์",
      "target_roles": "Data Analyst",
      "skills": "Python:L2;SQL:L2",
      "interests": "กีฬาฟุตบอล;ฟิสิกส์ภาษา",
      "onboard_grp": "Job_Hunter",
      "onboard_grp_description": "เตรียมฝึกงานสายเทคโนโลยี"
    }
  ],
  "response_time": "0.00000 s"
}
```

Response headers

```
alt-svc: h3="443"; ma=2592000,h3-29="443"; ma=2592000
content-length: 488
```

HyDE Feed Recommendation - Future Work

HyDE

Hyde Feed and Course recommi

localhost:4000/docs#/

Hyde Feed and Course recommendtation

1.1.0

OAS 3.1

[/openapi.json](#)

Hyde Feed and Course recommendation (In progress krub)

Last time Update : 2026-01-27 11:45:32

Repo :

[Contact Tun Kedsaro](#)

Health & Metadata

GET / Root Status

GET /health/ Health Check

GET /health/gemini Gemini Health Check

GET /health/bigquery Bigquery Health Check

Hyde Generator

GET /hyde/students/batch Generate Batch Recommendations

POST /hyde/students/{student_id} Generate Student Recommendation

Fetch results

POST /hyde/students/{student_id}/feed Get Student Feed

1:21 PM

2/3/2026

HyDE feed recommendation – Future Work

```
gcs.delete_by_ttl(  
    bucket_name = "hyde-datalake",  
    prefix="student001/",  
    ttl=timedelta(minutes=5)      # 5 min  
)  
  
gcs.delete_by_ttl(  
    bucket_name = "hyde-datalake",  
    prefix="student001/",  
    ttl=timedelta(hours=24)      # 24 hours ~ 1 day  
)  
  
gcs.delete_by_ttl(  
    bucket_name = "hyde-datalake",  
    prefix="student001/",  
    ttl=timedelta(days=30)      # 30 days ~ 1 month  
)  
  
gcs.delete_by_ttl(  
    bucket_name = "hyde-datalake",  
    prefix="student001/",  
    ttl=timedelta(weeks=48)      # 48 weeks ~ 1 years  
)
```

+

Create a Cloud Scheduler job

- Define the schedule

Name *
gotnojob-scheduler-trigger
Must be unique across the jobs in the same region

Region *
us-central1 (Iowa)

Frequency *
Schedules are specified using unix-cron format. E.g. every minute: '* * * * *', every 3 hours: '0 */3 * * *', every Monday at 9:00 a.m.: '0 9 * * 1'. [Learn more](#)

Time zone *
Coordinated Universal Time (UTC)

i Jobs set in timezones affected by daylight saving time can run outside of cadence during DST change. Using a UTC timezone can avoid the problem. [Learn more](#)

CONTINUE

- Configure the execution

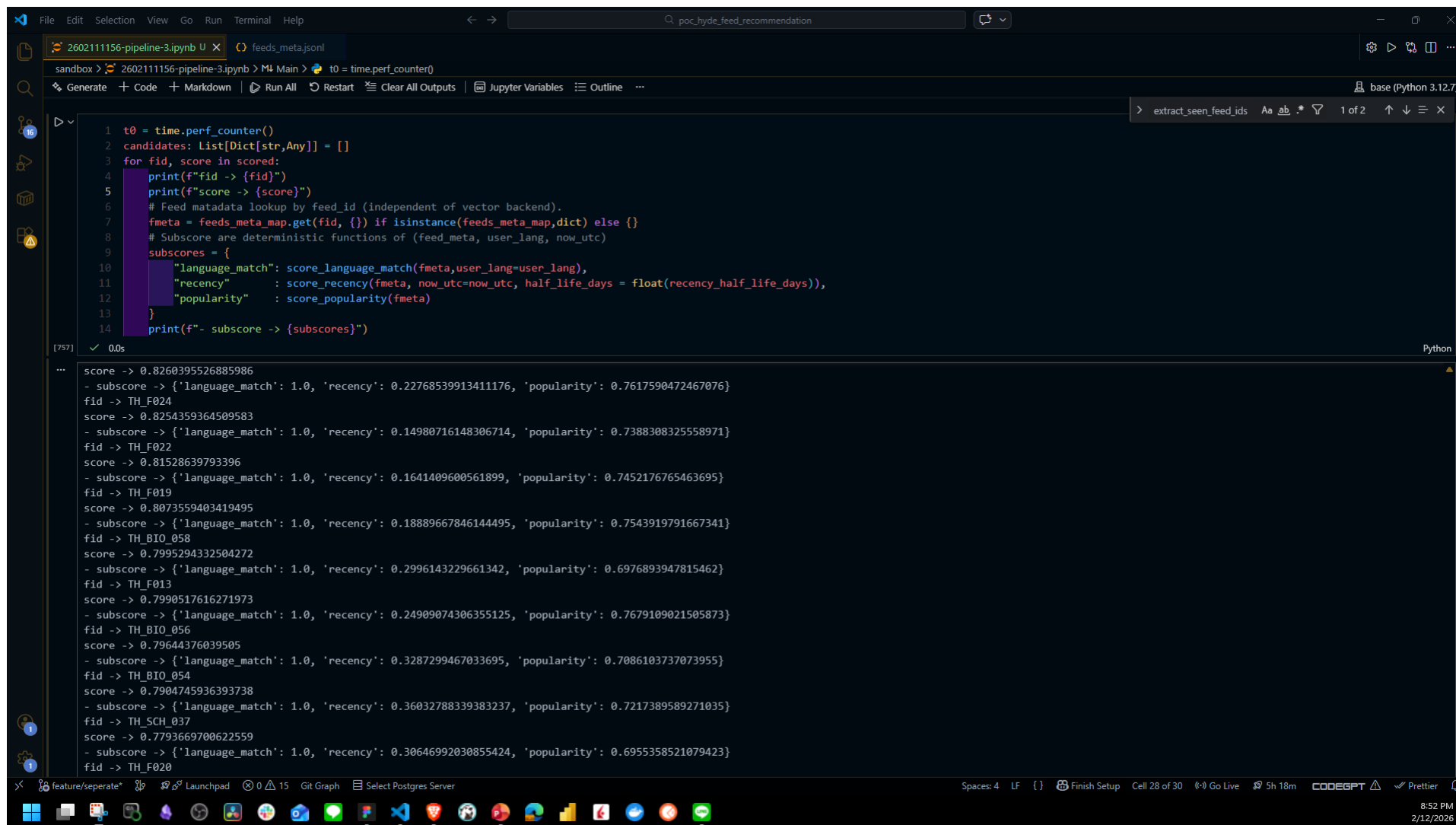
CREATE CANCEL

Team stand up

Thu, 12/02/26

- Done
 - Draft of subscore function
- Todo
 - Recheck pipeline code
 - Convert to API or .py something
- Issue
 - -

HyDE Feed Recommendation – Batch student HyDE generator



The screenshot displays a Jupyter Notebook environment with a dark theme. The top bar shows the file name '2602111156-pipeline-3.ipynb' and the current cell is 'Main'. The notebook is running on a 'base (Python 3.12.7)' kernel. The code in the cell is as follows:

```
1 t0 = time.perf_counter()
2 candidates: List[Dict[str,Any]] = []
3 for fid, score in scored:
4     print(f"fid -> {fid}")
5     print(f"score -> {score}")
6     # Feed metadata lookup by feed_id (independent of vector backend).
7     fmeta = feeds_meta_map.get(fid, {}) if isinstance(feeds_meta_map,dict) else {}
8     # Subscore are deterministic functions of (feed_meta, user_lang, now_utc)
9     subscores = {
10         "language_match": score_language_match(fmeta,user_lang=user_lang),
11         "recency"         : score_recency(fmeta, now_utc=now_utc, half_life_days = float(recency_half_life_days)),
12         "popularity"      : score_popularity(fmeta)
13     }
14     print(f"- subscore -> {subscores}")
```

The output of the code is displayed below the cell, showing a series of 'score' and 'subscore' outputs for various feed IDs. The output is as follows:

```
[757] ✓ 0.0s Python
...
score -> 0.8260395526885986
- subscore -> {'language_match': 1.0, 'recency': 0.22768539913411176, 'popularity': 0.7617590472467076}
fid -> TH_F024
score -> 0.8254359364509583
- subscore -> {'language_match': 1.0, 'recency': 0.14980716148306714, 'popularity': 0.7388308325558971}
fid -> TH_F022
score -> 0.81528639793396
- subscore -> {'language_match': 1.0, 'recency': 0.1641409600561899, 'popularity': 0.7452176765463695}
fid -> TH_F019
score -> 0.8073559403419495
- subscore -> {'language_match': 1.0, 'recency': 0.18889667846144495, 'popularity': 0.7543919791667341}
fid -> TH_BIO_058
score -> 0.7995294332504272
- subscore -> {'language_match': 1.0, 'recency': 0.2996143229661342, 'popularity': 0.6976893947815462}
fid -> TH_F013
score -> 0.7990517616271973
- subscore -> {'language_match': 1.0, 'recency': 0.24909074306355125, 'popularity': 0.7679109021505873}
fid -> TH_BIO_056
score -> 0.79644376039505
- subscore -> {'language_match': 1.0, 'recency': 0.3287299467033695, 'popularity': 0.7086103737073955}
fid -> TH_BIO_054
score -> 0.7904745936393738
- subscore -> {'language_match': 1.0, 'recency': 0.36032788339383237, 'popularity': 0.7217389589271035}
fid -> TH_SCH_037
score -> 0.7793669700622559
- subscore -> {'language_match': 1.0, 'recency': 0.30646992030855424, 'popularity': 0.6955358521079423}
fid -> TH_F020
```

The bottom status bar shows the notebook is running on a 'feature/separate*' branch, with 15 lines of code. The system clock indicates 8:52 PM on 2/12/2026.

Tue, 100226

- Done
 - Update 3 case handler retrieval support with hyde with FastAPI -> Deploy
- Todo
 - Update GCS Retrieval with handling 3 cases with FastAPI
 - HyDE generator
 - Batch HyDE student generator
 - Single HyDE student generator
- Issue
 - -

HyDE Feed Recommendation – Batch student HyDE generator

The image shows two side-by-side browser windows. The left window displays a web application for generating student recommendations, and the right window shows the Google Cloud Storage console.

Left Window: HyDE Feed and Course recommender

POST /hyde/students/{student_id} Generate Student Recommendation

API:02 Connectivity health check for Gemini LLM service. Verifies API availability and measures round-trip response latency.

Parameters

Name	Description
student_id * required	
any (path)	stu_p100

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:4000/hyde/students/stu_p100' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

http://localhost:4000/hyde/students/stu_p100

Server response

Code	Details
200	Response body <pre>{ "student_id": "stu_p100", "response": "complete" }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: * content-length: 47 content-type: application/json</pre>

Right Window: Google Cloud Storage

Cloud Storage **Bucket details**

hyde-datalake

Location: asia-southeast1 (Singapore) Storage class: Standard Public access: Not public Protection: Soft delete

Objects

Folder browser

hyde-datalake

stu_p001/

Filter by name prefix only **Filter**

No rows to display

Your folder is ready. Just add data.

Drop files and folders here or use the upload button. To move a lot of data from another bucket or cloud storage provider, create a transfer job.

Upload files **Create transfer job**

hyde-datalake/stu_p100/ deleted

HyDE Feed Recommendation - Batch student HyDE generator

The image displays two side-by-side screenshots from a Windows desktop environment.

The left screenshot shows the Google Cloud Storage console in a web browser. The page title is "Bucket details" for a bucket named "hyde-datalake". The location is "asia-southeast1 (Singapore)", storage class is "Standard", public access is "Not public", and protection is "Soft delete". The "Objects" tab is selected, showing a folder browser view of the bucket. The folder structure includes "hyde-datalake" and a series of subfolders named "stu_p000/" through "stu_p100/". The "Filter by name prefix only" option is selected, and the "Filter" button is visible. The "Rows per page" is set to 50, and the current view shows 1 - 12 of 12 items.

The right screenshot shows a web interface for a "Hyde Feed and Course recommender" service. The "Execute" button is highlighted. The "Responses" section shows a successful response (200) with the following details:

- Request URL:** `https://hyderecommend-service-dev-du7ytkyaqq-as-a.run.app/hyde/students/batch`
- Server response:** 200
- Response body:**

```
{  "student_id": [    "stu_p100",    "stu_p100",    "stu_p100",    "stu_p007",    "stu_p006",    "stu_p001",    "stu_p009",    "stu_p003",    "stu_p004",    "stu_p005",    "stu_p006",    "stu_p007",    "stu_p008",    "stu_p009",    "stu_p010",    "stu_p100"  ],  "response": "Complete"}
```
- Response headers:**

```
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
content-length: 181
content-type: application/json
date: Tue, 10 Feb 2026 11:02:03 GMT
server: Google Frontend
x-cloud-trace-context: 6f277c95a47ddaabc7e18738d5693b8b;o=1
```

The bottom of the right screenshot shows a table with the following data:

Code	Description	Links
200	Successful Response	No links

The system clock at the bottom right indicates the time is 6:02 PM on 2/10/2026.

HyDE Feed Recommendation – Single student HyDE generator

The image displays two side-by-side browser windows. The left window shows the Google Cloud Storage console for the bucket 'hyde-datalake'. The 'Objects' tab is active, showing a folder browser with subfolders 'stu_p001/', 'stu_p002/', and 'stu_p100/'. The right window shows a REST client interface for a POST request to the endpoint '/hyde/students/{student_id}'. The request body is a JSON object with 'student_id' set to 'stu_p002'. The response is a 200 status code with a JSON body containing 'student_id': 'stu_p002' and 'response': 'Complete'.

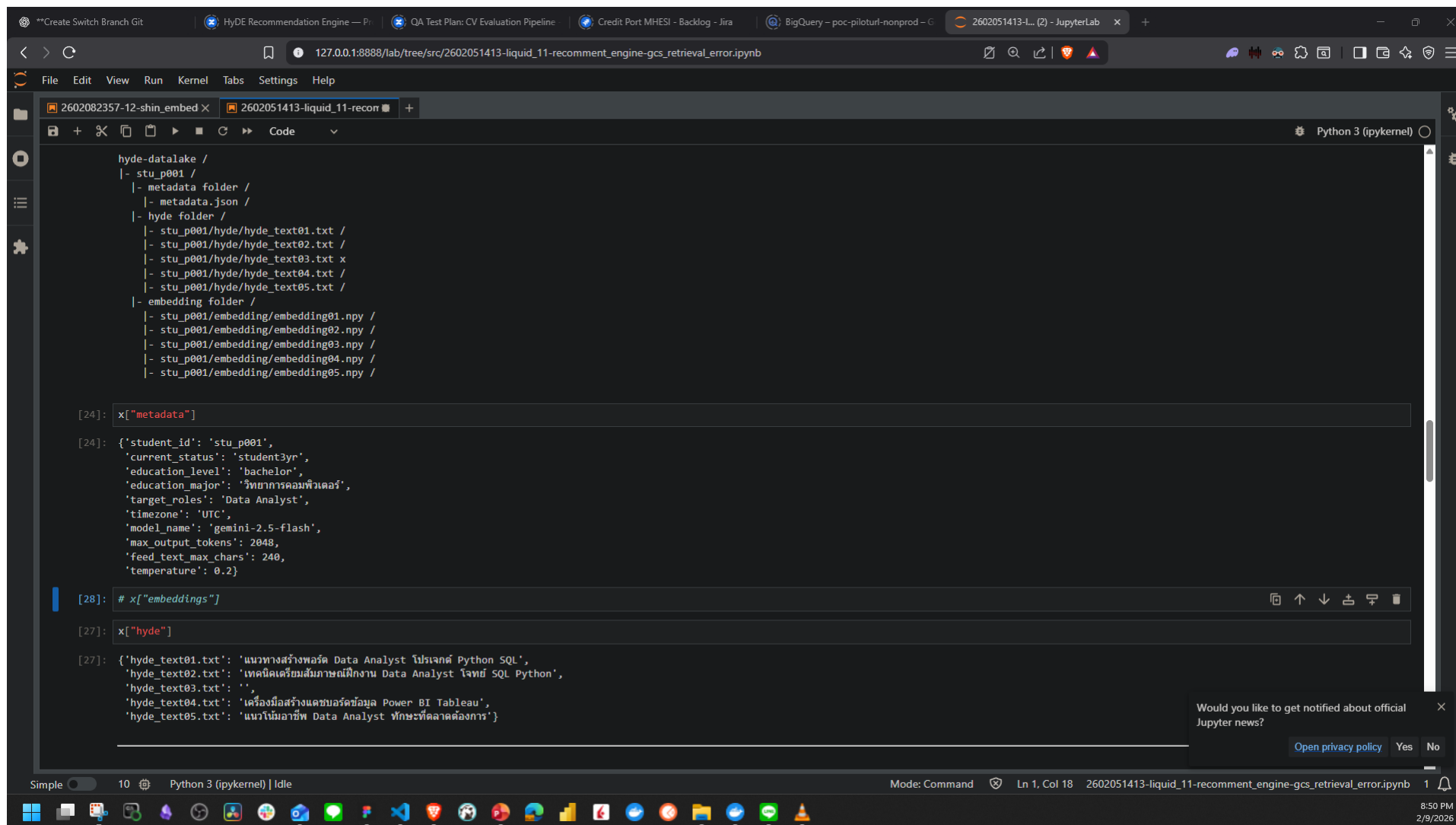
Google Cloud Storage Console (Left Window):

- URL: console.cloud.google.com/storage/
- Bucket: poc-piloturl-nonprod
- Bucket details for 'hyde-datalake':
 - Location: asia-southeast1 (Singapore)
 - Storage class: Standard
 - Public access: Not public
 - Protection: Soft delete
- Objects tab: Folder browser shows 'hyde-datalake' containing 'stu_p001/', 'stu_p002/', and 'stu_p100/'.

REST Client (Right Window):

- Method: POST
- Endpoint: /hyde/students/{student_id} Generate Student Recommendation
- Parameters: student_id (required) = stu_p002
- Execute button: Execute
- Responses: 200
- Response body: { "student_id": "stu_p002", "response": "Complete" }
- Response headers: access-control-allow-credentials: true, access-control-allow-origin: *, alt-svc: h3=":443"; ma=2592000, h3-29=":443"; ma=2592000, content-length: 47

HyDE Feed Recommendation - Single student HyDE generator



The screenshot displays a JupyterLab environment with the following components:

- Browser Tabs:** Multiple tabs are open, including "HyDE Recommendation Engine", "QA Test Plan: CV Evaluation Pipeline", "Credit Port MHESI - Backlog - Jira", "BigQuery - poc-piloturl-nonprod - G", and "2602051413-L... (2) - JupyterLab".
- Address Bar:** The URL is `127.0.0.1:8888/lab/tree/src/2602051413-liquid_11-recommend_engine-gcs_retrieval_error.ipynb`.
- File Tree:** The left sidebar shows a file structure:
 - `hyde-datalake /`
 - `|- stu_p001 /`
 - `|- metadata folder /`
 - `|- metadata.json /`
 - `|- hyde folder /`
 - `|- stu_p001/hyde/hyde_text01.txt /`
 - `|- stu_p001/hyde/hyde_text02.txt /`
 - `|- stu_p001/hyde/hyde_text03.txt x`
 - `|- stu_p001/hyde/hyde_text04.txt /`
 - `|- stu_p001/hyde/hyde_text05.txt /`
 - `|- embedding folder /`
 - `|- stu_p001/embedding/embedding01.npy /`
 - `|- stu_p001/embedding/embedding02.npy /`
 - `|- stu_p001/embedding/embedding03.npy /`
 - `|- stu_p001/embedding/embedding04.npy /`
 - `|- stu_p001/embedding/embedding05.npy /`

- Code Editor:** The main area contains Python code and terminal output:

```
hyde-datalake /
|- stu_p001 /
  |- metadata folder /
    |- metadata.json /
  |- hyde folder /
    |- stu_p001/hyde/hyde_text01.txt /
    |- stu_p001/hyde/hyde_text02.txt /
    |- stu_p001/hyde/hyde_text03.txt x
    |- stu_p001/hyde/hyde_text04.txt /
    |- stu_p001/hyde/hyde_text05.txt /
  |- embedding folder /
    |- stu_p001/embedding/embedding01.npy /
    |- stu_p001/embedding/embedding02.npy /
    |- stu_p001/embedding/embedding03.npy /
    |- stu_p001/embedding/embedding04.npy /
    |- stu_p001/embedding/embedding05.npy /

[24]: x["metadata"]

[24]: {'student_id': 'stu_p001',
      'current_status': 'student3yr',
      'education_level': 'bachelor',
      'education_major': 'วิทยาการคอมพิวเตอร์',
      'target_roles': 'Data Analyst',
      'timezone': 'UTC',
      'model_name': 'gemini-2.5-flash',
      'max_output_tokens': 2048,
      'feed_text_max_chars': 240,
      'temperature': 0.2}

[28]: # x["embeddings"]

[27]: x["hyde"]

[27]: {'hyde_text01.txt': 'แนวทางสร้างพอร์ต Data Analyst โปรแกรม Python SQL',
      'hyde_text02.txt': 'เทคนิคเตรียมสัมภาษณ์งาน Data Analyst โจทย์ SQL Python',
      'hyde_text03.txt': '',
      'hyde_text04.txt': 'เครื่องมือสร้างแดชบอร์ดข้อมูล Power BI Tableau',
      'hyde_text05.txt': 'แนวโน้มอาชีพ Data Analyst ที่กะทัดรัดต้องการ'}
```
- Terminal:** The bottom of the code editor shows the execution of the code cells.
- Notification:** A small notification in the bottom right corner asks: "Would you like to get notified about official Jupyter news?" with buttons for "Open privacy policy", "Yes", and "No".
- Status Bar:** The bottom of the window shows "Simple", "10", "Python 3 (ipykernel) | Idle", "Mode: Command", "Ln 1, Col 18", "2602051413-liquid_11-recommend_engine-gcs_retrieval_error.ipynb", and "1".
- Taskbar:** The Windows taskbar at the very bottom shows various application icons.

HyDE Feed Recommendation - System Architecture

```
{
  "student_id": "stu_p001",
  "status": ""
}
hyde-datalake /
|- stu_p001 /
  |- metadata folder /
    |- metadata.json /
  |- hyde folder /
    |- stu_p001/hyde/hyde_text01.txt /
    |- stu_p001/hyde/hyde_text02.txt /
    |- stu_p001/hyde/hyde_text03.txt x
    |- stu_p001/hyde/hyde_text04.txt /
    |- stu_p001/hyde/hyde_text05.txt /
  |- embedding folder /
    |- stu_p001/embedding/embedding01.npy /
    |- stu_p001/embedding/embedding02.npy /
    |- stu_p001/embedding/embedding03.npy /
    |- stu_p001/embedding/embedding04.npy /
    |- stu_p001/embedding/embedding05.npy /
    ...
  ...
"metadata": {
  "student_id": "stu_p001",
  "current_status": "student3yr",
  "education_level": "bachelor",
  "education_major": "วิทยาการคอมพิวเตอร์",
  "target_roles": "Data Analyst",
  "timezone": "UTC",
  "model_name": "gemini-2.5-flash",
  "max_output_tokens": 2048,
  "feed_text_max_chars": 240,
  "temperature": 0.2
},
"hyde": {
  "hyde_context1": "แนวทางสร้างพอร์ต Data Analyst โปแลนด์ Python SQL",
  "hyde_context2": "เทคนิคเตรียมสัมภาษณ์งาน Data Analyst ใจพุด SQL Python",
  "hyde_context3": "",
  "hyde_context4": "เครื่องมือสร้างแดชบอร์ดข้อมูล Power BI Tableau",
  "hyde_context5": "แนวโน้มอาชีพ Data Analyst ทักษะที่ตลาดต้องการ"
},
"embedded_vector": {
  "embedding_vector1": [
    0.018953053280711174,
    -0.009550142101943493,
    0.005990269593894482
  ],
  "embedding_vector2": [
    0.007405612617731094,
    -0.03062768094241619,
    0.0034155375324189663
  ],
  "embedding_vector3": [
    0.015558813698589802,
    0.006979266181588173,
    0.0075002312660217285
  ],
  "embedding_vector4": [
    0.001261499710381031,
    0.019617965444922447,
    0.012513278052210808
  ],
  "embedding_vector5": [
    -0.0200442373752594,
    0.004771450534462929,
    -0.00016345674521289766
  ]
}
}
```

Mon,090226

- Done

- Align CVEvaluation QA with P'bloom and update confluence
- Align Feed recommendation (*pipeline with flowchart)
 - Functions/core/subscore.py
 - Pipeline_3_gold_validate.py
- Align with shin about How vector embedding should be
- Plan tasks for Sprint 11 in Jira

- Todo

- Update GCS Retrieval class with new 3 conditions
- Develop pipeline to Update Embedding vector with shin format style
- unde data for each student (Pipeline_2_user_hyde_refresh.py)

- Issue

- -

HyDE Feed Recommendation - System Architecture & Technology Stack

```
1 ### case 2.1 :
2 bucket
3 |- embedding
4   |- embedding01.npy
5   |- embedding02.npy
6   |- embedding03.npy
7   |- embedding04.npy
8   |- embedding05.npy
9 |- metadata
10 |- metadata.json

1 ### case 2.2 :
2 bucket
3 |- embedding
4   |- embedding01.npy
5   |- embedding02.npy
6   |- embedding03.npy
7   |- embedding04.npy
8   |- embedding05.npy
9 |- metadata
10 |- metadata.json

1 ### case 2.3 :
2 bucket
3 |- embedding
4   |- embedding01.npy
5   |- embedding02.npy
6   |- embedding03.npy
7   |- embedding04.npy
8   |- embedding05.npy
9 |- metadata
10 |- metadata.json

1 ### case 2.4 :
2 bucket
3 |- embedding
4   |- embedding01.npy
5   |- embedding02.npy
6   |- embedding03.npy
7   |- embedding04.npy
8   |- embedding05.npy
9 |- metadata
10 |- metadata.json

1 ### case 2.5 :
2 bucket
3 |- embedding
4   |- embedding01.npy
5   |- embedding02.npy
6   |- embedding03.npy
7   |- embedding04.npy
8   |- embedding05.npy
9 |- metadata
10 |- metadata.json

1 ### case 3 :
2 bucket
3 |- embedding
4   |- embedding01.npy
5   |- embedding02.npy
6   |- embedding03.npy
7   |- embedding04.npy
8   |- embedding05.npy
9 |- metadata
10 |- metadata.json
```

HyDE Feed Recommendation - System Architecture & Technology Stack

The screenshot displays the system architecture and technology stack for the HyDE Feed Recommendation system, consisting of two main components: Google Cloud Storage and JupyterLab.

Google Cloud Storage (Left Panel):

- Bucket details:** The bucket is named `hyde-datalake`, located in `asia-southeast1 (Singapore)`, using `Standard` storage class, with `Not public` access and `Soft delete` protection.
- Objects:** The folder browser shows the path `hyde-datalake > stu_p001 > embedding/`. The folder contains four files: `embedding01.npy`, `embedding02.npy`, `embedding03.npy`, and `embedding04.npy`.
- Metadata:** A message at the bottom indicates `hyde-datalake/stu_p001/metadata/ deleted`.

JupyterLab (Right Panel):

- Code Execution:** The code in the JupyterLab cell shows the bucket exists and lists the metadata and embedding files. A warning message is displayed: `UserWarning: BigQuery Storage module not found, fetch data with the REST endpoint instead.`
- Output:** The output shows the folder structure and the contents of the `embedding` folder, including the `embedding01.npy` file.

System Architecture & Technology Stack:

- Cloud Storage:** Google Cloud Storage (GCS) bucket `hyde-datalake`.
- Storage Class:** Standard.
- Access:** Not public.
- Protection:** Soft delete.
- Location:** asia-southeast1 (Singapore).
- Folder Structure:** `hyde-datalake/stu_p001/embedding/`.
- Files:** `embedding01.npy`, `embedding02.npy`, `embedding03.npy`, `embedding04.npy`.
- Metadata:** `hyde-datalake/stu_p001/metadata/` (deleted).
- JupyterLab:** Python 3 (ipykernel) environment.
- Code:** `Bucket exists : hyde-datalake`, `- metadata_prefix -> stu_p001/metadata`, `- metadata_path -> stu_p001/metadata/metadata.json`, `activate query data from bigquery function ...`, `/usr/local/lib/python3.12/site-packages/google/cloud/bigquery/table.py:1994: UserWarning: BigQuery Storage module not found, fetch data with the REST endpoint instead.`, `warnings.warn()`, `- embedding_prefix -> stu_p001/embedding`.
- Output:** `hyde-datalake /`, `|- stu_p001 /`, `|- metadata folder x`, `|- metadata.json x`, `|- embedding folder /`, `|- stu_p001/embedding/embedding01.npy /`, `|- stu_p001/embedding/embedding02.npy /`, `|- stu_p001/embedding/embedding03.npy x`, `|- stu_p001/embedding/embedding04.npy /`, `|- stu_p001/embedding/embedding05.npy x`.

HyDE Feed Recommendation - System Architecture & Technology Stack

The screenshot displays two side-by-side windows. The left window is the Google Cloud Storage console, showing the 'hyde-datalake' bucket details. The bucket is located in 'asia-southeast1 (Singapore)' and uses the 'Standard' storage class. The 'Objects' tab is selected, showing a folder browser for 'hyde-datalake' with subfolders 'embedding/' and 'metadata/'. A message at the bottom indicates that 'hyde-datalake/stu_p001/metadata/metadata.json' has been deleted.

The right window is a JupyterLab interface running a Python 3 (ipykernel) session. The code in the editor defines a bucket structure and lists its contents. The output shows the bucket exists and lists the metadata and embedding folders. The embedding folder contains five files: 'embedding01.npy', 'embedding02.npy', 'embedding03.npy', 'embedding04.npy', and 'embedding05.npy'. The output also shows the contents of the 'embedding01.npy' file, which is an array of five float32 values.

```
print()
print(x['status'])

Bucket exists : hyde-datalake
- metadata_prefix -> stu_p001/metadata
- metadata_path -> stu_p001/metadata/metadata.json
- embedding_prefix -> stu_p001/embedding

hyde-datalake /
|- stu_p001 /
  |- metadata folder /
    |- metadata.json x
  |- embedding folder /
    |- stu_p001/embedding/embedding01.npy /
    |- stu_p001/embedding/embedding02.npy /
    |- stu_p001/embedding/embedding03.npy x
    |- stu_p001/embedding/embedding04.npy /
    |- stu_p001/embedding/embedding05.npy x

[202]: x["metadata"]

[202]: {}

[204]: x["embeddings"]

[204]: {'embedding01.npy': array([ 0.01895305, -0.00955014,  0.00599027], dtype=float32),
      'embedding02.npy': array([ 0.00509049, -0.04070203, -0.0031905 ], dtype=float32),
      'embedding03.npy': array([], dtype=float64),
      'embedding04.npy': array([ 0.01695731,  0.01713481, -0.01191018], dtype=float32),
      'embedding05.npy': array([], dtype=float64)}
```

case 1 : happy

```
### case 1 :
bucket      /
|- embedding /
  |- embedding01.npy /
  |- embedding02.npy /
```

HyDE Feed Recommendation - System Architecture & Technology Stack

The screenshot displays two side-by-side windows illustrating the system architecture and technology stack for HyDE Feed Recommendation.

Left Window: Google Cloud Storage Console

- Bucket details:** The bucket is named `hyde-datalake`, located in `asia-southeast1 (Singapore)` with `Standard` storage class, `Not public` access, and `Soft delete` protection.
- Objects:** The folder browser shows the hierarchy: `hyde-datalake` > `stu_p001/` > `embedding/` > `metadata/`. The `embedding/` folder contains files like `embedding01.npy`, `embedding02.npy`, and `embedding04.npy`.
- Notification:** A message at the bottom states: "hyde-datalake/stu_p001/embedding/embedding05.npy deleted".

Right Window: JupyterLab Interface

- Code Editor:** The code defines the bucket structure and lists the files in the `embedding` folder. The structure is as follows:

```
hyde-datalake /
|- stu_p001 /
|  |- metadata folder /
|  |  |- metadata.json /
|  |  |- embedding folder /
|  |     |- stu_p001/embedding/embedding01.npy /
|  |     |- stu_p001/embedding/embedding02.npy /
|  |     |- stu_p001/embedding/embedding03.npy x
|  |     |- stu_p001/embedding/embedding04.npy /
|  |     |- stu_p001/embedding/embedding05.npy x
```
- Output:** The code execution results show the status of the bucket and the metadata of the student record. The output for `x["metadata"]` is:

```
{'student_id': 'stu_p001',
 'current_status': 'student3yr',
 'education_level': 'bachelor',
 'education_major': 'วิทยาการคอมพิวเตอร์',
 'target_roles': 'Data Analyst',
 'timezone': 'UTC',
 'model_name': 'gemini-2.5-flash',
 'max_output_tokens': 2048,
 'feed_text_max_chars': 240,
 'temperature': 0.2}
```

System Architecture & Technology Stack:

- Cloud Storage:** Google Cloud Storage (GCS) bucket named `hyde-datalake`.
- Storage Location:** `asia-southeast1 (Singapore)`.
- Storage Class:** `Standard`.
- Access:** `Not public`.
- Protection:** `Soft delete`.
- Folder Structure:** `hyde-datalake` > `stu_p001/` > `embedding/` > `metadata/`.
- Files:** `embedding01.npy`, `embedding02.npy`, `embedding03.npy`, `embedding04.npy`, `embedding05.npy`.
- Notification:** `hyde-datalake/stu_p001/embedding/embedding05.npy deleted`.
- Code Execution:** JupyterLab (Python 3 (ipykernel)) running the code to interact with the bucket.
- Output:** The code execution results show the status of the bucket and the metadata of the student record.

HyDE Feed Recommendation - System Architecture & Technology Stack

The image displays two side-by-side screenshots illustrating the system architecture and technology stack for HyDE Feed Recommendation.

Left Screenshot: Google Cloud Storage Console

- Bucket details:** The bucket is named `hyde-datalake`, located in `asia-southeast1 (Singapore)` with `Standard` storage class, `Not public` access, and `Soft delete` protection.
- Folder browser:** The view shows the hierarchy: `hyde-datalake` > `stu_p001/` > `embedding/` > `metadata/`. The `embedding/` folder contains five files: `embedding01.npy`, `embedding02.npy`, `embedding03.npy`, `embedding04.npy`, and `embedding05.npy`.
- Notification:** A message at the bottom indicates `hyde-datalake/stu_p001/metadata/ deleted`.

Right Screenshot: JupyterLab Notebook

- Code:** The notebook shows a script for interacting with the bucket. It defines the bucket name `hyde-datalake`, the metadata prefix `stu_p001/metadata`, and the embedding prefix `stu_p001/embedding`. It lists the contents of the `stu_p001/` folder, including the metadata folder and the embedding folders.
- Output:** The notebook displays the output of the script, showing the metadata and embeddings for a specific student.
- Metadata:** The output shows a dictionary for `"metadata"` with fields: `student_id` (`'stu_p001'`), `current_status` (`'student3yr'`), `education_level` (`'bachelor'`), `education_major` (`'วิทยาการคอมพิวเตอร์'`), `target_roles` (`'Data Analyst'`), `timezone` (`'UTC'`), `model_name` (`'gemini-2.5-flash'`), `max_output_tokens` (`2048`), `feed_text_max_chars` (`240`), and `temperature` (`0.2`).
- Embeddings:** The output shows a dictionary for `"embeddings"` with five entries, each containing an array of float values and the dtype `float32`.

Fri,060226

- Done

- Align CVEvaluation QA with P'bloom and update confluence
- Align Feed recommendation (*pipeline with flowchart)
 - Functions/core/subscore.py
 - Pipeline_3_gold_validate.py
- Align with shin about How vector embedding should be
- Plan tasks for Sprint 11 in Jira

- Todo

- Update GCS Retrieval class with new 3 conditions
- Develop pipeline to Update Embedding vector with shin format style
- unde data for each student (Pipeline_2_user_hyde_refresh.py)

- Issue

- -

Thu,050226

- Done
 - Read and understood the QA Test Plan for the CV Evaluation Pipeline link
 - Understanding task for sprint 11 then prepare material to discuss (B4 P'bloom vacation)
- Todo
 - Align CVEvaluation QA with P'bloom
 - Align Feed recommendation (*pipeline with flowchart)
 - Functions/core/subscore.py
 - Pipeline_3_gold_validate.py
 - Align with shin about How vector embedding should be
 - Plan tasks for Sprint 11 in Jira
 - Update GCS Retrieval class with new 3 conditions
 - Develop pipeline to bUpdate Embedding vector with shin format style
 - unde data for each student (Pipeline_2_user_hyde_refresh.py)
- Issue
 - Question in Slack
 - Question in Figma

Wed,040226

- Done
 - Sprint review deck for Sprint 10
 - Develop an endpoint to query data from Cloud Storage
- Todo
 - Read and understood the QA Test Plan for the CV Evaluation Pipeline link
 - Plan tasks for Sprint 11 in Jira
 - Develop pipeline to bundle data for each student (Pipeline_2_user_hyde_refresh.py)
- Issue
 - -

```
{
  "student_id": "stu_p003",
  "metadata": {
    "student_id": "stu_p003",
    "current_status": "student4+yr",
    "education_level": "bachelor",
    "education_major": "สถิติ",
    "target_roles": "Data Analyst",
    "timezone": "UTC",
    "model_name": "gemini-2.5-flash",
    "max_output_tokens": 2048,
    "feed_text_max_chars": 240,
    "temperature": 0.2
  },
  "embedded_vector": {
    "emb1": [
      0.0003576562739908695,
      ...,
      -0.02179572731256485,
    ],
    "emb2": [
      0.0003576562739908695,
      ...,
      -0.02179572731256485,
    ],
    "emb3": [
      0.0003576562739908695,
      ...,
      -0.02179572731256485,
    ],
    "emb4": [
      0.0003576562739908695,
      ...,
      -0.02179572731256485,
    ],
    "emb5": [
      0.0003576562739908695,
      ...,
      -0.02179572731256485,
    ]
  }
}
```

Tue, 05/02/26

- Done
 - Read and understood the QA Test Plan for the CV Evaluation Pipeline link
- Todo
 - Plan tasks for Sprint 11 in Jira
 - Develop pipeline to bundle data for each student (Pipeline_2_user_hyde_refresh.py)
- Issue
 - -

HyDE Feed Recommendation - System Overview

Case 1 : ∞ budget + metadata

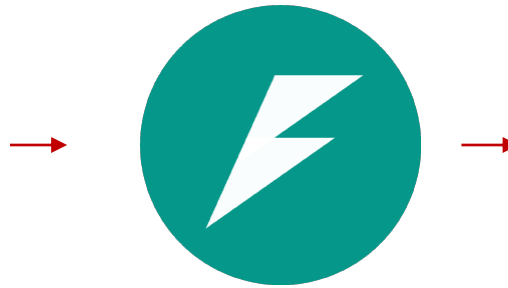
Case 2 : Storage(budget x metadata) -> return Only metadata

Case 3 : Have no budget -> return data from bigquery

If case 2 or case 3 -> talk with shin what going on?

If some vector is missing or not complete return only complete vector and other "error"

```
hyde-datalake-feeds/  
├─ stu_p001/  
│   ├── metadata/  
│   │   └─ metadata.json  
│   └─ embedding/  
│       ├── embedding01.npy  
│       ├── embedding02.npy  
│       ├── embedding03.npy  
│       ├── embedding04.npy  
│       └─ embedding05.npy  
├─ stu_p002/  
├─ ...  
└─ stu_p010/
```



```
{  
  "student_id": "stu_p003",  
  "metadata": {  
    "student_id": "stu_p003",  
    "current_status": "student4+yr",  
    "education_level": "bachelor",  
    "education_major": "สถิติ",  
    "target_roles": "Data Analyst",  
    "timezone": "UTC",  
    "model_name": "gemini-2.5-flash",  
    "max_output_tokens": 2048,  
    "feed_text_max_chars": 240,  
    "temperature": 0.2  
  },  
  "embedded_vector": {  
    "emb1": [  
      0.0003576562739908695,  
      ...  
      -0.02179572731256485,  
    ],  
    "emb2": [  
      0.0003576562739908695,  
      ...  
      -0.02179572731256485,  
    ],  
    "emb3": [  
      0.0003576562739908695,  
      ...  
      -0.02179572731256485,  
    ],  
    "emb4": [  
      0.0003576562739908695,  
      ...  
      -0.02179572731256485,  
    ],  
    "emb5": [  
      0.0003576562739908695,  
      ...  
      -0.02179572731256485,  
    ]  
  }  
}
```

Tue, 03/02/26

- Done
 - Prepared sprint review deck for Sprint 10
- Todo
 - Implement item retirement on Cloud Storage (code + cron job)
 - Plan tasks for Sprint 11 in Jira
 - Develop pipeline to bundle data for each student
 - Read and understood the QA Test Plan for the CV Evaluation Pipeline link

Process of work

1. POC -> MVP
 - POC : all full pipeline code
 - MVP : POC + production plan + owner
2. Estimate cost man-day and grant chart

- 1 Sprint = 2 weeks
- We have sprint grooming before sprint start
- We use Jira to estimate timeline and ticket
- There are ghostdeck
- There are confluence for project detail
- There are POC
- Estimate manday cost
- Design database
- Architecture
- RACI
- SIT เหลืออีก 4 เดือน เดือน june
- UAT เหลืออีก 6 เดือน เดือน สิงหาคม
- Golive เหลืออีก ... เดือน เดือน October